



Trường Công Nghệ và Thiết Kế

KHOA CÔNG NGHỆ THÔNG TIN KINH DOANH

Báo Cáo Đồ Án

TÌM HIỂU INCREMENTAL DBSCAN

Học phần: MÁY HỌC

Mã học phần: 25D1INF50904402

Giảng viên: NGUYỄN AN TẾ

Chuyên ngành: KHOA HỌC MÁY TÍNH

LỜI NÓI ĐẦU

Trong bối cảnh dữ liệu ngày càng gia tăng nhanh chóng cả về quy mô lẫn tính chất biến đổi liên tục, việc nghiên cứu và ứng dụng các phương pháp học máy nhằm khai thác, phân tích và trích xuất thông tin từ dữ liệu trở nên vô cùng cần thiết. Đặc biệt, các bài toán phân cụm dữ liệu đóng vai trò quan trọng trong việc khám phá cấu trúc tiềm ẩn của dữ liệu mà không cần đến nhãn trước. Trong số các thuật toán phân cụm, DBSCAN là một phương pháp tiêu biểu nhờ khả năng phát hiện các cụm có hình dạng bất kỳ và xử lý tốt dữ liệu nhiễu. Tuy nhiên, hạn chế lớn của DBSCAN truyền thống là phải thực hiện lại toàn bộ quá trình phân cụm khi dữ liệu thay đổi, gây tốn kém chi phí tính toán trong các hệ thống dữ liệu động.

Xuất phát từ thực tiễn đó, đề án tập trung nghiên cứu và triển khai thuật toán Incremental DBSCAN, một hướng tiếp cận cải tiến cho phép cập nhật kết quả phân cụm một cách hiệu quả khi dữ liệu được bổ sung theo thời gian. Thông qua việc xây dựng mô hình, cài đặt thuật toán và tiến hành thực nghiệm trên dữ liệu, đề án hướng đến việc đánh giá khả năng ứng dụng của phương pháp này trong các bài toán thực tế.

Trong quá trình thực hiện đề án, nhóm chúng em đã nhận được sự hướng dẫn tận tình và những kiến thức quý báu từ TS.Nguyễn An Tế. Nhóm chúng em xin gửi lời cảm ơn chân thành đến thầy vì đã hỗ trợ, định hướng và tạo điều kiện thuận lợi để chúng em có thể hoàn thành tốt đề tài này. Đồng thời, chúng em cũng xin chân thành cảm ơn Đại học Kinh tế TP. Hồ Chí Minh đã cung cấp môi trường học tập chuyên nghiệp, tạo nền tảng kiến thức vững chắc và điều kiện thuận lợi cho sinh viên trong suốt quá trình học tập và nghiên cứu.

Mặc dù đã cố gắng hoàn thiện đề án một cách tốt nhất, tuy nhiên do hạn chế về thời gian, kinh nghiệm cũng như kiến thức chuyên môn, bài báo cáo khó tránh khỏi những thiếu sót nhất định. Chúng em rất mong nhận được sự góp ý và nhận xét từ giảng viên để có thể tiếp tục hoàn thiện và nâng cao kiến thức của mình trong thời gian tới.

MỤC LỤC

LỜI NÓI ĐẦU	2
MỤC LỤC HÌNH ẢNH.....	6
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI	7
I. Giới thiệu đề tài.....	7
II. Mục tiêu nghiên cứu	8
III. Phương pháp nghiên cứu.....	9
IV. Cấu trúc đồ án	10
V. Tài nguyên sử dụng	10
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	11
I. DBSCAN	11
1. Giới thiệu về DBSCAN.....	11
2. Các khái niệm và Tham số cốt lõi	11
3. Cách thức hoạt động	12
4. Ưu điểm và nhược điểm	15
II. Incremental DBSCAN.....	16
1. Định nghĩa và mục tiêu.....	16
2. So sánh với DBSCAN truyền thống.....	16
3. Chiến lược xử lý dữ liệu mới.....	17
4. Xử lý điểm bị xóa	17
CHƯƠNG 3. CÁC BƯỚC GIẢI BÀI TOÁN ĐƠN GIẢN.....	19
I. Giới thiệu bài toán minh họa đơn giản.....	19
II. Xác định các tham số Eps và MinPts	20
1. Xác định MinPts	20
2. Các bước tìm Eps theo phương pháp K-Distance:	21
3. Phân loại Core Point, Border Point, Noise	24
4. Thêm điểm mới và cập nhật cụm (Incremental).....	26
5. Kết quả phân cụm	27
CHƯƠNG 4: TRIỂN KHAI INCREMENTAL DBSCAN	29

I. Giới thiệu bộ dữ liệu	29
II. Giới thiệu bộ dữ liệu	29
III. Cài đặt Incremental DBSCAN bằng Python.....	30
IV. Biểu diễn kết quả	34
V. Đánh giá hiệu suất.....	36
CHƯƠNG 5: SO SÁNH INCREMENTAL DBSCAN VỚI CÁC THUẬT TOÁN PHÂN CỤM KHÁC	38
I. Giới thiệu chung.....	38
II. Các thuật toán được sử dụng trong so sánh	38
1. K-Means Clustering.....	38
2. Hierarchical Agglomerative Clustering (HAC).....	39
3. DBSCAN truyền thống.....	39
4. Incremental DBSCAN	40
III. Phân tích chi tiết các tiêu chí đánh giá	40
1. Khả năng xử lý dữ liệu luôn thay đổi (Dynamic Data Handling)	40
2. Hiệu suất phân cụm (Clustering Quality)	41
3. Khả năng xử lý dữ liệu nhiễu (Noise Handling).....	43
4. Độ phức tạp tính toán (Computational Complexity)	43
5. Thời gian cập nhật dữ liệu (Update Time)	43
IV. Phân tích kết quả thực nghiệm	44
V. Kết luận	44
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	47
I. Tổng kết kết quả đạt được.....	47
II. Hạn chế và thách thức	48
III. Đề xuất cải tiến.....	48
IV. Ứng dụng thực tế.....	49
TÀI LIỆU THAM KHẢO	51
BẢNG PHÂN CÔNG NHIỆM VỤ	53

MỤC LỤC HÌNH ẢNH

Figure 1. Sơ đồ khối thuật toán DBSCAN, nhấn vào đây để truy cập.....	14
Figure 2. Sơ đồ khối thuật toán Incremental DBSCAN, nhấn vào đây để truy cập.....	18
Figure 3. Điểm và tọa độ (x,y)	19
Figure 4. Biểu đồ phân tán (Scatter Plot)	20
Figure 5. Bảng phân tích và xác định khoảng cách giữa các điểm	22
Figure 6. Bảng xác định khoảng cách k láng giềng.....	23
Figure 7. Bảng thứ tự giảm dần khoảng cách.....	23
Figure 8. Biểu đồ K-Distance với $k = 3$	24
Figure 9. Bảng xét điểm lân cận xung quanh C1	25
Figure 10. Bảng xét điểm lân cận xung quanh C5	25
Figure 11. Bảng phân cụm cuối cùng cho bài toán phân cụm khách hàng	26
Figure 12. Tiến hành phân cụm xét theo điểm mới C9	27
Figure 13. Kết quả phân cụm dựa theo bài toán phân cụm khách hàng.....	28
Figure 14. Minh họa cấu trúc khởi tạo của thuật toán Incremental DBSCAN	30
Figure 15. Minh họa việc xây dựng KD-Tree từ dữ liệu hiện tại.....	31
Figure 16. Minh họa truy vấn các điểm trong bán kính eps.....	31
Figure 17. Minh họa quá trình đưa dữ liệu vào hệ thống theo từng batch	32
Figure 18. Minh họa quá trình mở rộng cụm từ một điểm lõi.....	32
Figure 19. Minh họa cơ chế hợp nhất cụm khi một điểm kết nối nhiều cụm.....	33
Figure 20. Minh họa việc gán nhãn phân cụm vào tập dữ liệu	34
Figure 21. Minh họa kết quả phân cụm của Incremental DBSCAN trên dữ liệu moons	34
Figure 22. So sánh thời gian xử lý giữa DBSCAN và Incremental DBSCAN	35
Figure 23. Minh họa kết quả phân cụm của các thuật toán khác nhau.....	36
Figure 24. Minh họa hiệu suất phân cụm	42

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

I. Giới thiệu đề tài

1. Giới thiệu tổng quan về DBSCAN và Incremental DBSCAN

Trong lĩnh vực học máy không giám sát, bài toán phân cụm dữ liệu đóng vai trò quan trọng trong việc khám phá cấu trúc tiềm ẩn của dữ liệu mà không cần nhãn. Một trong những thuật toán tiêu biểu trong nhóm này là DBSCAN (Density-Based Spatial Clustering of Applications with Noise), được xây dựng dựa trên ý tưởng mật độ. Thuật toán này xác định các cụm dữ liệu bằng cách tìm các vùng có mật độ điểm cao, đồng thời phân biệt rõ ràng giữa các điểm thuộc cụm và các điểm nhiễu.

Khác với các phương pháp như K-Means vốn giả định cụm có dạng hình cầu, DBSCAN có khả năng phát hiện các cụm với hình dạng bất kỳ. Điều này đặc biệt phù hợp với các bộ dữ liệu phi tuyến tính, điển hình như bộ dữ liệu dạng “hai nửa mặt trăng” được sử dụng trong đồ án này. Ngoài ra, DBSCAN còn có ưu điểm là không cần xác định trước số lượng cụm và có khả năng xử lý nhiễu hiệu quả thông qua việc gán nhãn -1 cho các điểm không thuộc bất kỳ cụm nào.

Tuy nhiên, DBSCAN truyền thống được thiết kế để xử lý dữ liệu theo cách tĩnh, nghĩa là toàn bộ tập dữ liệu phải được đưa vào cùng một thời điểm để tiến hành phân cụm. Khi dữ liệu mới xuất hiện, thuật toán không có cơ chế cập nhật mà buộc phải thực hiện lại toàn bộ quá trình từ đầu. Điều này làm phát sinh chi phí tính toán lớn và không phù hợp với các hệ thống dữ liệu hiện đại, nơi dữ liệu liên tục được bổ sung theo thời gian.

Để khắc phục hạn chế này, Incremental DBSCAN được phát triển như một phiên bản mở rộng của DBSCAN, cho phép cập nhật kết quả phân cụm một cách linh hoạt khi dữ liệu thay đổi. Thay vì xử lý toàn bộ dữ liệu cùng lúc, thuật toán này tiếp nhận dữ liệu theo từng phần nhỏ và điều chỉnh các cụm hiện có dựa trên thông tin mới. Trong đồ án này, Incremental DBSCAN được triển khai với cơ chế xử lý theo batch kết hợp với cấu trúc KD-Tree nhằm tối ưu hóa truy vấn lân cận, đồng thời áp dụng chiến lược hợp nhất

cụm mạnh (strong merging) để đảm bảo tính nhất quán của kết quả phân cụm trong quá trình cập nhật.

2. Lý do chọn đề tài

Trong thực tế, dữ liệu thường không tồn tại dưới dạng tĩnh mà liên tục được sinh ra và cập nhật theo thời gian. Các hệ thống như thương mại điện tử, mạng xã hội, hệ thống theo dõi hành vi người dùng hay các ứng dụng IoT đều tạo ra dữ liệu với tốc độ cao và khối lượng lớn. Điều này đặt ra yêu cầu cấp thiết đối với các thuật toán học máy phải có khả năng xử lý dữ liệu động một cách hiệu quả.

Mặc dù DBSCAN là một thuật toán phân cụm mạnh và linh hoạt, nhưng việc phải tính toán lại từ đầu mỗi khi dữ liệu thay đổi khiến nó trở nên kém hiệu quả trong các môi trường thực tế. Đặc biệt, với các tập dữ liệu lớn như trong đồ án này với 15,000 điểm dữ liệu, chi phí tính toán sẽ tăng đáng kể nếu áp dụng DBSCAN theo cách truyền thống.

Chính vì vậy, việc nghiên cứu và triển khai Incremental DBSCAN trở nên cần thiết nhằm giải quyết bài toán cập nhật dữ liệu một cách hiệu quả. Thuật toán này không chỉ giúp giảm thời gian tính toán mà còn cho phép hệ thống phản ứng nhanh hơn với sự thay đổi của dữ liệu. Ngoài ra, việc so sánh Incremental DBSCAN với các thuật toán phổ biến khác như K-Means và HAC trên cùng một bộ dữ liệu phi tuyến tính cũng giúp làm rõ ưu và nhược điểm của từng phương pháp trong các tình huống khác nhau.

II. Mục tiêu nghiên cứu

Mục tiêu chính của đồ án là nghiên cứu và làm rõ nguyên lý hoạt động của thuật toán Incremental DBSCAN, từ đó triển khai và đánh giá hiệu quả của thuật toán này qua bộ dữ liệu giả lập. Trước hết, đồ án tập trung vào việc hiểu rõ cơ chế hoạt động của DBSCAN truyền thống, bao gồm các khái niệm như điểm lõi, điểm biên, điểm nhiễu, cũng như cách thức hình thành và mở rộng cụm dựa trên mật độ.

Tiếp theo, đồ án tiến hành xây dựng và triển khai thuật toán Incremental DBSCAN bằng ngôn ngữ Python, với việc thiết kế các cấu trúc dữ liệu phù hợp để lưu

trữ dữ liệu động và cập nhật cụm theo từng batch. Việc sử dụng KD-Tree trong quá trình truy vấn lân cận giúp cải thiện hiệu suất tính toán, trong khi chiến lược hợp nhất cụm đảm bảo rằng các cụm không bị phân mảnh khi dữ liệu mới được thêm vào.

Bên cạnh đó, đồ án cũng hướng đến việc đánh giá hiệu suất của thuật toán thông qua các chỉ số như Silhouette Score, Davies-Bouldin Index và Calinski-Harabasz Index, đồng thời đo lường thời gian thực thi để so sánh với DBSCAN truyền thống. Cuối cùng, việc so sánh với các thuật toán khác như K-Means và HAC giúp đưa ra cái nhìn toàn diện về hiệu quả của Incremental DBSCAN trong các bài toán phân cụm dữ liệu phi tuyến tính.

III. Phương pháp nghiên cứu

Đồ án sử dụng phương pháp nghiên cứu kết hợp giữa lý thuyết và thực nghiệm nhằm đảm bảo tính toàn diện và chính xác trong đánh giá. Về mặt lý thuyết, các khái niệm và nguyên lý của DBSCAN và Incremental DBSCAN được phân tích dựa trên các tài liệu học thuật và giáo trình về học máy không giám sát.

Về mặt thực nghiệm, đồ án tiến hành xây dựng một bộ dữ liệu phi tuyến tính gồm 15,000 điểm dữ liệu bằng hàm `make_moons`, nhằm mô phỏng các cấu trúc dữ liệu phức tạp thường gặp trong thực tế. Dữ liệu sau đó được tiền xử lý bằng phương pháp chuẩn hóa (StandardScaler) để đảm bảo tính đồng nhất trước khi đưa vào các thuật toán phân cụm.

Thuật toán Incremental DBSCAN được triển khai thủ công thông qua việc xây dựng các hàm xử lý như truy vấn lân cận, mở rộng cụm và cập nhật nhãn, đồng thời sử dụng KD-Tree để tối ưu hiệu suất. Song song với đó, các thuật toán DBSCAN truyền thống, K-Means và HAC cũng được triển khai để làm cơ sở so sánh.

Kết quả của các thuật toán được đánh giá dựa trên các chỉ số định lượng và được trực quan hóa thông qua các biểu đồ nhằm hỗ trợ quá trình phân tích và rút ra kết luận.

IV. Cấu trúc đề án

Đề án được tổ chức thành sáu chương chính nhằm đảm bảo tính logic và dễ theo dõi. Chương đầu tiên bao gồm việc trình bày phần mở đầu với các nội dung giới thiệu, lý do chọn đề tài và mục tiêu nghiên cứu.

Chương thứ hai tập trung vào cơ sở lý thuyết của DBSCAN và Incremental DBSCAN, làm rõ các khái niệm và cơ chế hoạt động của thuật toán.

Chương thứ ba trình bày các bước giải bài toán phân cụm trên một ví dụ đơn giản nhằm minh họa cách thức hoạt động của thuật toán.

Chương thứ tư là phần triển khai thực nghiệm trên bộ dữ liệu thực tế, bao gồm các bước tiền xử lý, cài đặt thuật toán, trực quan hóa kết quả và đánh giá hiệu suất.

Chương thứ năm thực hiện so sánh giữa Incremental DBSCAN và các thuật toán khác dựa trên nhiều tiêu chí khác nhau.

Chương cuối cùng đưa ra kết luận, đánh giá tổng thể và đề xuất các hướng phát triển trong tương lai.

V. Tài nguyên sử dụng

Trong quá trình thực hiện đề án, các công cụ và thư viện phổ biến trong Python được sử dụng nhằm hỗ trợ việc xử lý dữ liệu và triển khai thuật toán. Cụ thể, thư viện NumPy và Pandas được sử dụng để thao tác và quản lý dữ liệu, trong khi Scikit-learn cung cấp các công cụ hỗ trợ cho việc chuẩn hóa dữ liệu, đánh giá mô hình và triển khai các thuật toán phân cụm như DBSCAN, K-Means và HAC.

Ngoài ra, thư viện Matplotlib và Seaborn được sử dụng để trực quan hóa dữ liệu và kết quả phân cụm, giúp quá trình phân tích trở nên trực quan và dễ hiểu hơn. Đặc biệt, cấu trúc KD-Tree được sử dụng thông qua thư viện Scikit-learn nhằm tối ưu hóa việc truy vấn lân cận trong quá trình triển khai Incremental DBSCAN.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

I. DBSCAN

1. Giới thiệu về DBSCAN

DBSCAN (Density-based spatial clustering of applications with noise) là một thuật toán phân cụm tiêu biểu trong học máy không giám sát, được thiết kế nhằm phát hiện các cụm dữ liệu dựa trên mật độ phân bố của các điểm trong không gian. Khác với các phương pháp phân cụm dựa trên phân hoạch như K-Means, DBSCAN không dựa vào việc tối ưu hóa khoảng cách đến tâm cụm, mà thay vào đó định nghĩa cụm như các vùng có mật độ điểm cao, được ngăn cách bởi các vùng mật độ thấp đại diện cho nhiễu hoặc các điểm ngoại lai.

Điểm đặc biệt của DBSCAN là khả năng:

- Tự động xác định số lượng cụm: Thuật toán không yêu cầu người dùng cung cấp trước tham số K (số lượng cụm)
- Nhận diện nhiễu hiệu quả: Có khả năng tách biệt các điểm không thuộc về bất kỳ cụm nào.
- Phát hiện hình dạng phức tạp: Có thể tìm thấy các cụm có hình dạng và kích thước bất kỳ mà các thuật toán dựa trên khoảng cách thông thường thường bỏ sót.

2. Các khái niệm và Tham số cốt lõi

Để mô hình hóa khái niệm mật độ trong không gian dữ liệu, DBSCAN sử dụng hai tham số quan trọng là ϵ (**Eps**) và **MinPts**. Hai tham số này đóng vai trò quyết định trong việc xác định cấu trúc cụm cũng như phân loại các điểm dữ liệu.

- **Epsilon (ϵ hay Eps):** Trước hết, với một điểm dữ liệu p , vùng lân cận ϵ -neighborhood của điểm này được định nghĩa là tập hợp tất cả các điểm q nằm

trong khoảng cách nhỏ hơn hoặc bằng ϵ . Về mặt toán học, vùng lân cận được biểu diễn như sau: $N_\epsilon(p) = \{q \in D \mid d(p, q) < \epsilon\}$.

- Để tối ưu việc tìm kiếm lân cận trong không gian nhiều chiều, cấu trúc dữ liệu KDTree được sử dụng giúp giảm độ phức tạp từ $O(n^2)$ xuống $O(n \log n)$.
- + Trong đó, D là tập dữ liệu và $d(p, q)$ là hàm khoảng cách giữa hai điểm p và q . Trong hầu hết các trường hợp, khoảng cách Euclidean được sử dụng:

$$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

- **MinPts**: là tham số biểu diễn số lượng điểm tối thiểu cần có trong vùng lân cận $N_\epsilon(p)$ để một điểm được xem là có mật độ đủ cao.

Phân loại các điểm dữ liệu: Dựa vào hai tham số trên, thuật toán chia các điểm thành 3 loại:

1. **Core Point (Điểm lõi):** Điểm c mà tại đó vùng lân cận ϵ chứa ít nhất $MinPts$ điểm.

$$\text{Điều kiện: } |N_\epsilon(c)| \geq MinPts$$

2. **Border Point (Điểm biên):** Điểm b không phải là điểm lõi ($|N_\epsilon(b)| < MinPts$) nhưng nằm trong vùng lân cận của một điểm lõi nào đó.

$$\text{Điều kiện: } |N_\epsilon(b)| < MinPts \wedge (\exists c \in N_\epsilon(b)) \mid c \text{ là điểm lõi}$$

3. **Noise Point (Điểm nhiễu):** Điểm không thỏa mãn cả hai điều kiện trên. Những điểm này không thuộc bất kỳ cụm nào.

3. Cách thức hoạt động

Để hình thành các cụm, DBSCAN sử dụng các khái niệm về khả năng tiếp cận (reachability):

Các định nghĩa về kết nối:

- **Direct density reachability (DDR - Tiếp cận mật độ trực tiếp):** Điểm q được coi là DDR từ p nếu $q \in N_\epsilon(p)$ và p là một điểm lõi. Ký hiệu $p \rightarrow q$. Tính chất này có tính chất bất đối xứng (asymmetric) nếu một điểm là biên.
- **Density reachability (DR - Tiếp cận mật độ):** Là một chuỗi các bước DDR. Nếu tồn tại một chuỗi $p_1, p_2, \dots, p_n$, với $p_1 = p$ và $p_n = q$ sao cho p_{i+1} là DDR từ p_i .
- **Density connectivity (DC - Kết nối mật độ):** Hai điểm p và q được coi là DC nếu có một điểm lõi o mà từ đó cả p và q đều có khả năng tiếp cận mật độ (DR). Đây là tính chất đối xứng (symmetric) và là cơ sở để gộp các điểm vào cùng một cụm.

Quy trình thực hiện:

1. **Gán nhãn ban đầu:** Tất cả các điểm được đánh dấu là chưa phân loại (UNCLASSIFIED).
2. **Quét dữ liệu:** Lặp qua từng điểm chưa phân loại. Nếu điểm đó là Core Point, thực hiện ExpandCluster.
3. **Mở rộng cụm (ExpandCluster):** Nếu gặp một điểm là Core Point, một cụm mới được khởi tạo.
 - Thuật toán sử dụng một hàng đợi (seeds) để tìm tất cả các điểm có khả năng tiếp cận mật độ từ điểm lõi này và gán chúng vào cùng một ClusterID.
 - Trong quá trình mở rộng, nếu gặp một điểm lõi khác, vùng lân cận của nó cũng được thêm vào hàng đợi để tiếp tục lan tỏa cụm.
4. **Xử lý nhiễu:** Nếu một điểm không thể mở rộng và không thuộc vùng lân cận của điểm lõi nào, nó bị gán nhãn NOISE.

Sơ đồ khối thuật toán DBSCAN:

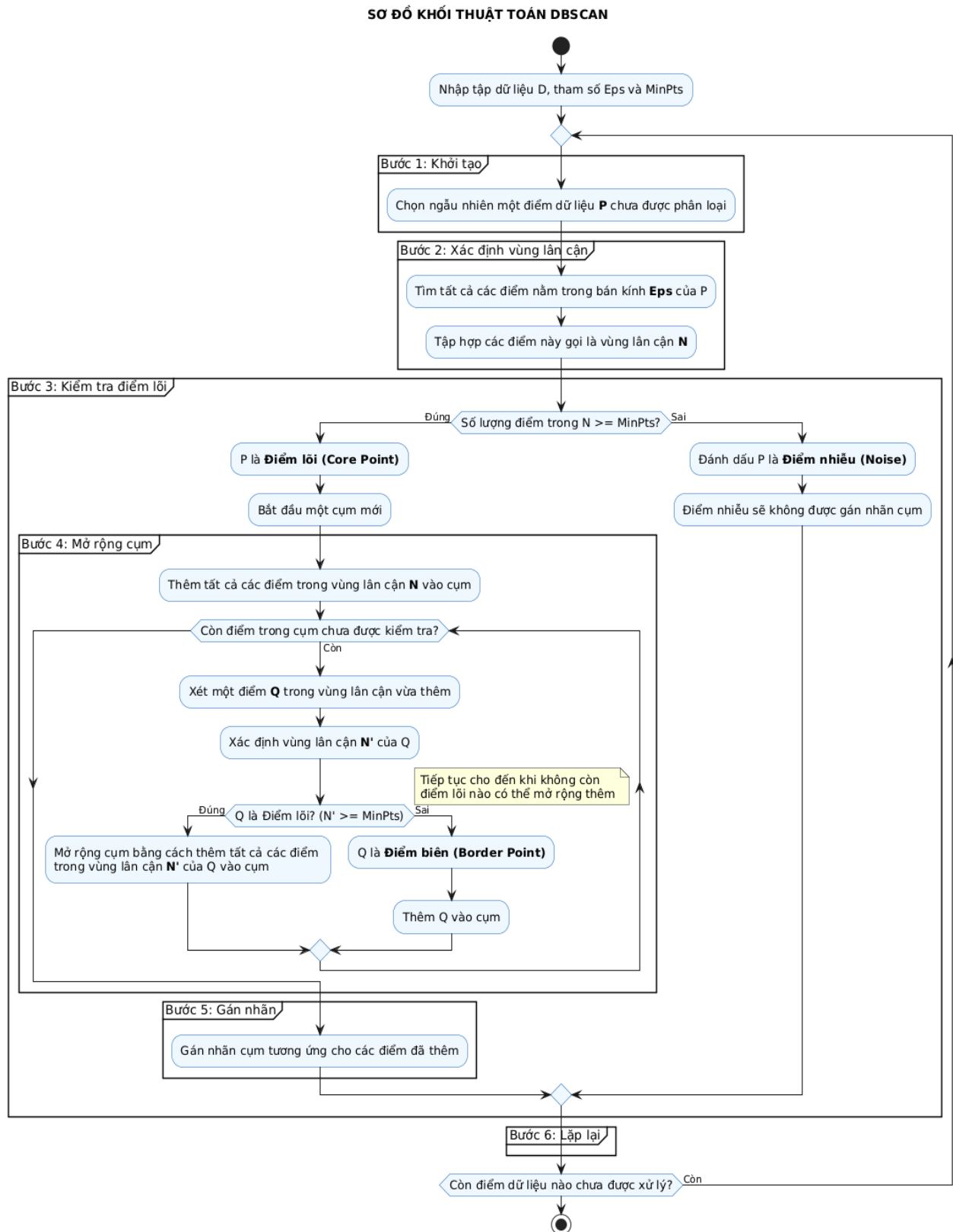


Figure 1. Sơ đồ khối thuật toán DBSCAN, nhấn vào [đây](#) để truy cập.

4. Ưu điểm và nhược điểm

Ưu điểm:

- Hình dạng bất kỳ: Khác với các thuật toán như K-Means vốn giả định các cụm có dạng hình cầu hoặc lõi, DBSCAN dựa trên mật độ để “lên lõi” theo cấu trúc dữ liệu, từ đó có thể phát hiện các cụm phi tuyến tính phức tạp. Điều này đặc biệt quan trọng trong các bài toán thực tế, nơi dữ liệu thường không tuân theo các phân bố hình học đơn giản. Ví dụ, với các tập dữ liệu có dạng vòng tròn, hình xoắn hoặc hai dải cong như dữ liệu “moons”, DBSCAN vẫn có thể phân cụm chính xác trong khi K-Means thường cho kết quả sai lệch.
- Kháng nhiễu: Tự động tách biệt các điểm rác, không ép buộc tất cả các điểm dữ liệu phải thuộc về một cụm, mà cho phép tồn tại các điểm nhiễu được gán nhãn riêng nhằm không để chúng làm sai lệch tâm cụm.
- Không cần số cụm K: Tự động hình thành cụm dựa trên mật độ, giúp đơn giản hóa quá trình triển khai và giảm phụ thuộc vào kinh nghiệm của người dùng từ đó giảm bớt gánh nặng phải thử sai số lượng cụm ban đầu.

Nhược điểm:

- Nhạy cảm tham số: Việc chọn ϵ và $MinPts$ không phù hợp sẽ làm kết quả thay đổi hoàn toàn. Hai tham số này ảnh hưởng trực tiếp đến việc xác định mật độ và ranh giới cụm. Nếu ϵ quá nhỏ, nhiều điểm sẽ bị coi là nhiễu do không đủ số lượng lân cận, dẫn đến việc cụm bị phân mảnh. Ngược lại, nếu ϵ quá lớn, các cụm khác nhau có thể bị gộp lại thành một cụm duy nhất. Tương tự, giá trị $MinPts$ không phù hợp cũng có thể làm sai lệch hoàn toàn kết quả phân cụm. Do đó, việc lựa chọn tham số thường đòi hỏi quá trình thử nghiệm và hiểu biết nhất định về dữ liệu.
- Mật độ không đồng nhất: Trong trường hợp dữ liệu chứa các cụm có mật độ rất khác nhau, việc sử dụng một cặp tham số $(\epsilon, MinPts)$ duy nhất sẽ không thể mô tả chính xác tất cả các cụm. Các cụm có mật độ thấp có thể bị xem là nhiễu, trong

khi các cụm có mật độ cao có thể bị gộp lại. Đây là một vấn đề phổ biến trong các tập dữ liệu thực tế, nơi phân bố dữ liệu thường không đồng đều.

- Dữ liệu lớn/Số chiều cao: Hiệu suất giảm khi số chiều tăng (do khoảng cách Euclidean trở nên kém hiệu quả trong không gian cao chiều). Điều này làm giảm hiệu quả của việc xác định vùng lân cận và dẫn đến kết quả phân cụm kém chính xác. Đồng thời, chi phí tính toán cũng tăng lên đáng kể do việc tính khoảng cách trong không gian nhiều chiều phức tạp hơn.

II. Incremental DBSCAN

1. Định nghĩa và mục tiêu

Incremental DBSCAN (IDBSCAN) là phiên bản cập nhật của DBSCAN dành cho các cơ sở dữ liệu động.

Định nghĩa: Thuật toán cho phép duy trì và điều chỉnh cấu trúc cụm tại thời điểm t khi tập dữ liệu D thay đổi, mà không cần thực hiện lại toàn bộ quá trình phân cụm từ đầu. Điều này giúp tận dụng kết quả đã có, chỉ tập trung xử lý những phần bị ảnh hưởng bởi sự thay đổi.

Mục tiêu: Cập nhật kết quả phân cụm sau mỗi lần thay đổi dữ liệu bằng cách tính toán trực tiếp trên các vùng bị ảnh hưởng thay vì chạy lại toàn bộ thuật toán từ đầu (re-clustering), từ đó giảm thiểu đáng kể thời gian và nỗ lực tính toán.

2. So sánh với DBSCAN truyền thống

Cơ chế xử lý: DBSCAN truyền thống là thuật toán tĩnh (static), coi dữ liệu là cố định. IDBSCAN là thuật toán động (dynamic/incremental), cập nhật theo thời gian thực.

Phạm vi tính toán: DBSCAN thực hiện trên toàn bộ n điểm dữ liệu ($O(n \log n)$ hoặc $O(n^2)$). IDBSCAN chỉ tính toán lại trong vùng N_ϵ của điểm thay đổi, giúp giảm độ phức tạp xuống mức phụ thuộc vào mật độ cục bộ.

3. Chiến lược xử lý dữ liệu mới

Khi thêm một điểm p vào tập dữ liệu D , sự thay đổi mật độ chỉ xảy ra trong phạm vi lân cận của điểm đó.

Khái niệm "Đối tượng bị ảnh hưởng" (Affected Objects): Bao gồm các điểm trong $N_\varepsilon(p)$ và tất cả các điểm có khả năng tiếp cận mật độ từ các điểm trong $N_\varepsilon(p)$.

Quy trình cập nhật:

1. **Xác định tập ảnh hưởng:** Chỉ các đối tượng q thỏa mãn $dist(p, q) \leq \varepsilon$ mới có thể thay đổi trạng thái (từ nhiều thành biên/lõi hoặc liên kết các cụm).
2. **Cập nhật mật độ:** $|N_\varepsilon(q)_{mới}| = |N_\varepsilon(q)_{cũ}| + 1$.
3. **Các trường hợp xảy ra:**
 - **Thành nhiều:** p không làm thay đổi trạng thái Core Point của bất kỳ điểm nào và bản thân nó cũng không phải Core Point.
 - **Mở rộng cụm:** p kết nối với một điểm lõi hiện tại và được hấp thụ vào cụm đó.
 - **Hợp nhất cụm (Merge):** Nếu p trở thành điểm lõi và trong lân cận của nó chứa các điểm thuộc về các cụm khác nhau ($C_1, C_2, \dots$), thuật toán thực hiện hợp nhất: $C_{new} = C_i \cup \{p\}$.
 - **Tạo cụm mới:** Nếu p làm cho một tập các điểm nhiều trước đó đủ điều kiện mật độ $|N_\varepsilon| \geq MinPts$.

4. Xử lý điểm bị xóa

Khi một điểm p bị xóa, quy trình diễn ra ngược lại:

1. **Giảm mật độ:** Với mỗi $q \in N_\varepsilon(p)$, thực hiện $|N_\varepsilon(q)_{mới}| = |N_\varepsilon(q)_{cũ}| - 1$.
2. **Thay đổi vai trò:** Nếu một điểm lõi q không còn thỏa mãn $MinPts$, nó bị hạ cấp xuống thành điểm biên hoặc nhiều.

3. **Tái cấu trúc cụm (Split):** Đây là phần quan trọng nhất. Nếu điểm p bị xóa (hoặc điểm lõi q bị hạ cấp) đóng vai trò là "cầu nối" mật độ duy nhất trong một cụm, thuật toán phải kiểm tra tính kết nối mật độ (Density Connectivity).
- Sử dụng thuật toán duyệt đồ thị (BFS/DFS) trong phạm vi cụm bị ảnh hưởng để kiểm tra xem các phần còn lại có còn đường đi qua các điểm lõi khác hay không.
 - Nếu không còn đường đi, cụm cũ bị chia tách thành hai hoặc nhiều cụm nhỏ. Các điểm không còn liên kết với điểm lõi nào sẽ bị gán nhãn NOISE.

Sơ đồ khối thuật toán Incremental DBSCAN:

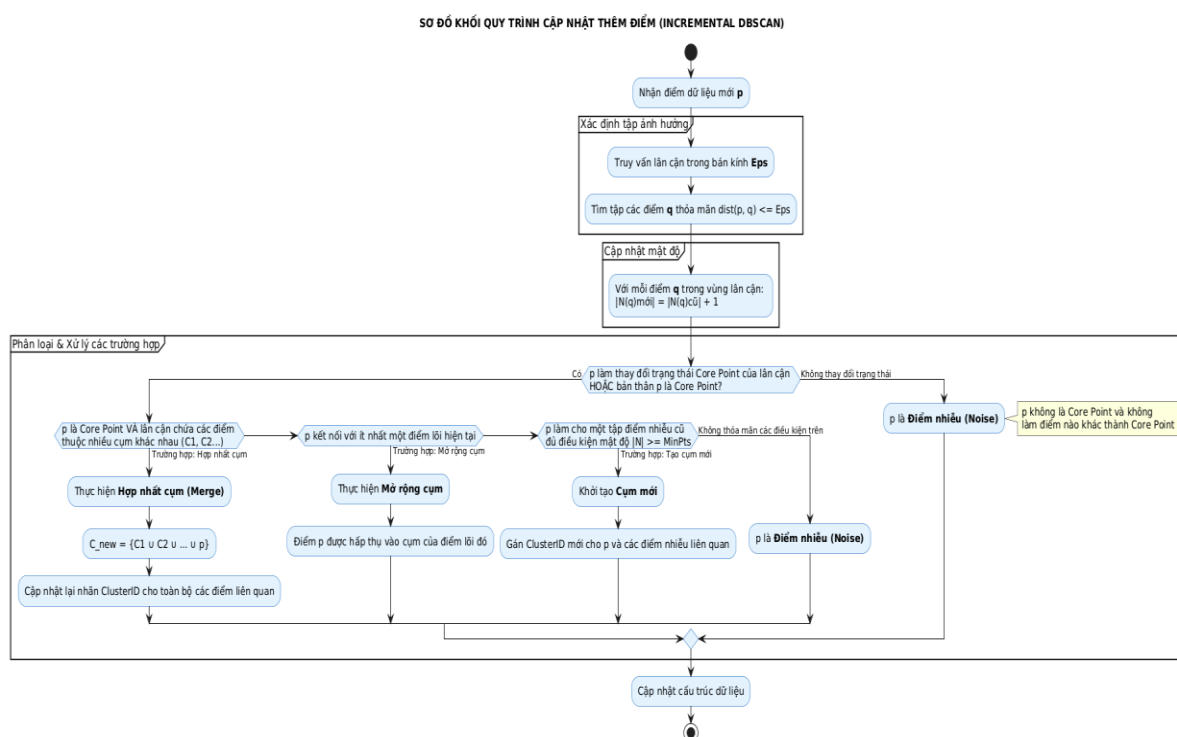


Figure 2. Sơ đồ khối thuật toán Incremental DBSCAN, nhấn vào [đây](#) để truy cập.

CHƯƠNG 3. CÁC BƯỚC GIẢI BÀI TOÁN ĐƠN GIẢN

I. Giới thiệu bài toán minh họa đơn giản

Để minh họa chi tiết từng bước tính toán và làm nổi bật ưu điểm cập nhật cục bộ của thuật toán Incremental DBSCAN, đồ án sử dụng một bài toán kinh điển trong lĩnh vực phân tích dữ liệu: **Bài toán phân khúc khách hàng** (Customer Segmentation).

- Dữ liệu của các khách hàng được biểu diễn trên không gian không gian 2 chiều (2D) với 2 đặc trưng cơ bản:
 - + Trục x: Thu nhập hàng năm (đơn vị tính: ngàn USD).
 - + Trục y: Điểm chi tiêu (thang điểm đánh giá từ 1 đến 100).

Giả sử tại thời điểm ban đầu (ký hiệu là thời điểm t_0), hệ thống cơ sở dữ liệu của siêu thị ghi nhận một tập hợp gồm 9 điểm dữ liệu (đại diện cho 9 khách hàng). Tọa độ của các khách hàng này được mô tả chi tiết trong bảng dưới đây:

Điểm	Tọa độ (x,y)
C1	(20,20)
C2	(22,22)
C3	(22,20)
C4	(20,22)
C5	(80,90)
C6	(80,92)
C7	(82,90)
C8	(82,92)
O1	(100,10)

Figure 3. Điểm và tọa độ (x,y)

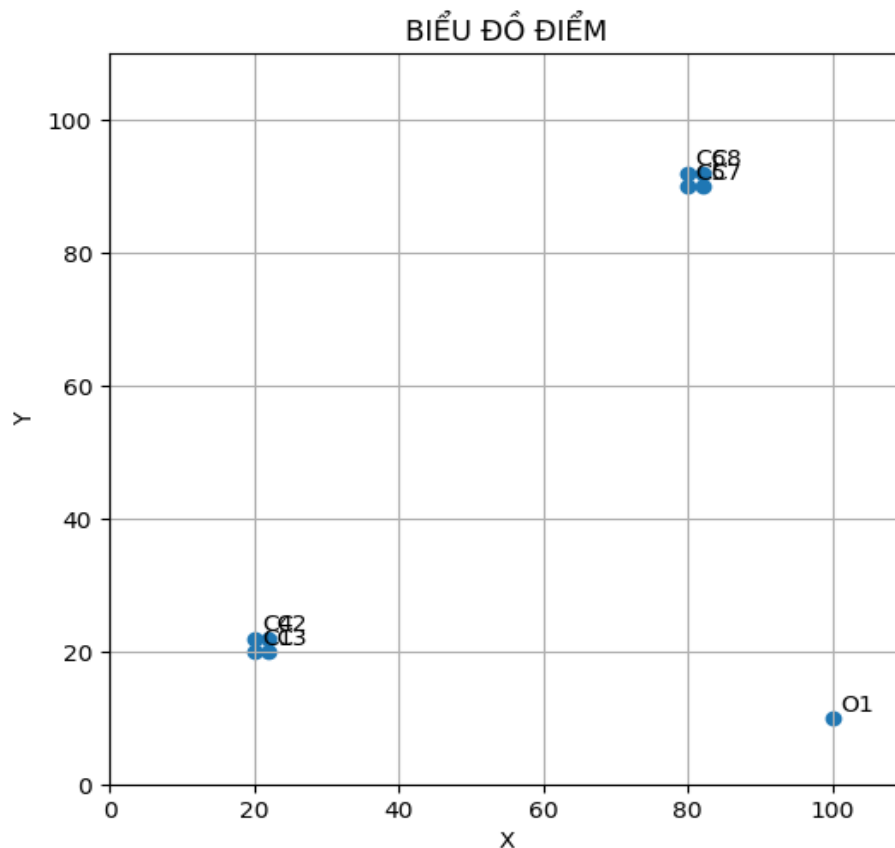


Figure 4. Biểu đồ phân tán (Scatter Plot)

Mục tiêu là sử dụng thuật toán DBSCAN để phân cụm dữ liệu, dựa vào hai tham số chính: Eps (bán kính lân cận) và MinPts (số điểm tối thiểu để tạo thành cụm).

II. Xác định các tham số Eps và MinPts

Để thuật toán Incremental DBSCAN phân cụm tập dữ liệu, ta cần thiết lập hai tham số đầu vào là MinPts (số điểm tối thiểu) và Eps (bán kính lân cận).

1. Xác định MinPts

Xác định tham số MinPts: Theo DBSCAN, có 2 công thức để xác định giá trị MinPts dựa trên số chiều (dim) của tập dữ liệu:

- Công thức 1: ($\text{MinPts} = \text{dim} + 1$). Đây là ngưỡng tối thiểu về mặt hình học để tạo thành một cấu trúc không gian (ví dụ: không gian 2 chiều cần ít nhất 3 điểm).

- Công thức 2: ($\text{MinPts} \geq 2 \times \text{dim}$). Đây là quy tắc thực nghiệm (Rule of Thumb) được khuyến nghị bởi các chuyên gia để áp dụng cho dữ liệu thực tế.

Lý do KHÔNG chọn Công thức 1: Công thức 1 yêu cầu mật độ quá lỏng lẻo, khiến thuật toán trong việc công nhận một điểm là Điểm lõi quá đơn giản. Việc này dẫn đến hai rủi ro lớn:

- Khả năng kháng nhiễu kém: Thuật toán dễ bị đánh lừa bởi các điểm rác vô tình nằm gần nhau và gom chúng thành một cụm sai lệch.
- Hiện tượng "bắc cầu" (Chaining Effect): Các cụm khách hàng có hành vi riêng biệt dễ bị nối lại thành một cụm không lồ chỉ vì một vài điểm nhiễu lọt vào giữa làm cầu nối. Do đó, để đảm bảo phân tách ranh giới phân khúc khách hàng rõ ràng và lọc nhiễu tốt nhất, đề án loại bỏ Công thức 1 và chọn Công thức 2.

=> Sử dụng công thức số 2

- Trong bài toán này, dữ liệu có 2 chiều (tọa độ x, y), nên MinPts hợp lý nhất là $\text{MinPts} \geq 2 - 2 = 4$.

2. Các bước tìm Eps theo phương pháp K-Distance:

Bước 1: Áp dụng công thức tính khoảng cách Euclide để tính khoảng cách giữa các điểm.

$$d(A(x_1, y_1), B(x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

	C1	C2	C3	C4	C5	C6	C7	C8	O1
C1	0	2.828	2	2	92.195	93.723	93.509	95.016	80.623
C2	2.828	0	2	2	89.376	90.907	90.686	92.195	78.918
C3	2	2	0	2.828	90.907	92.455	92.195	93.723	78.638
C4	2	2	2.828	0	90.686	92.195	92.022	93.509	80.895

C5	92.195	89.376	90.907	90.686	0	2	2	2.828	82.462
C6	93.723	90.907	92.455	92.195	2	0	2.828	2	84.404
C7	93.509	90.686	92.195	92.022	2	2.828	0	2	82.000
C8	95.016	92.195	93.723	93.509	2.828	2	2	0	83.952
O1	80.623	78.918	78.638	80.895	82.462	84.404	82.000	83.952	0

Figure 5. Bảng phân tích và xác định khoảng cách giữa các điểm

Bước 2: Xác định khoảng cách k-láng giềng:

Chọn giá trị $k = \text{MinPts} - 1 = 4 - 1 = 3 \Rightarrow$ Chọn $k = 3$. Với mỗi điểm, sắp xếp khoảng cách từ nhỏ đến lớn và ghi lại khoảng cách từ điểm đó đến điểm gần nhất thứ 3 (vì $k = 3$).

- C1: (C3: 2), (C4: 2), (C2: 2.828),... $\Rightarrow$ Chọn 2.828
- C2: (C3: 2), (C4: 2), (C1: 2.828),... $\Rightarrow$ Chọn 2.828
- C3: (C1: 2), (C2: 2), (C4: 2.828),... $\Rightarrow$ Chọn 2.828
- C4: (C1: 2), (C2: 2), (C3: 2.828),... $\Rightarrow$ Chọn 2.828
- C5: (C6: 2), (C7: 2), (C8: 2.828),... $\Rightarrow$ Chọn 2.828
- C6: (C5: 2), (C8: 2), (C7: 2.828),... $\Rightarrow$ Chọn 2.828
- C7: (C5: 2), (C8: 2), (C6: 2.828),... $\Rightarrow$ Chọn 2.828
- C8: (C6: 2), (C7: 2), (C5: 2.828),... $\Rightarrow$ Chọn 2.828
- O1: (C3: 78.638), (C2: 78.918), (C1: 80.623),... $\Rightarrow$ Chọn 80.623

	C1	C2	C3	C4	C5	C6	C7	C8	O1
C1	0	2.828	2	2	92.195	93.723	93.509	95.016	80.623
C2	2.828	0	2	2	89.376	90.907	90.686	92.195	78.918
C3	2	2	0	2.828	90.907	92.455	92.195	93.723	78.638

C4	2	2	2.828	0	90.686	92.195	92.022	93.509	80.895
C5	92.195	89.376	90.907	90.686	0	2	2	2.828	82.462
C6	93.723	90.907	92.455	92.195	2	0	2.828	2	84.404
C7	93.509	90.686	92.195	92.022	2	2.828	0	2	82.000
C8	95.016	92.195	93.723	93.509	2.828	2	2	0	83.952
O1	80.623	78.918	78.638	80.895	82.462	84.404	82.000	83.952	0

Figure 6. Bảng xác định khoảng cách k láng giềng

Bước 3: Sắp xếp theo thứ tự giảm dần và vẽ biểu đồ K-Distance

Điểm	Khoảng cách
C1	2.828
C2	2.828
C3	2.828
C4	2.828
C5	2.828
C6	2.828
C7	2.828
C8	2.828
O1	80.623

Figure 7. Bảng thứ tự giảm dần khoảng cách

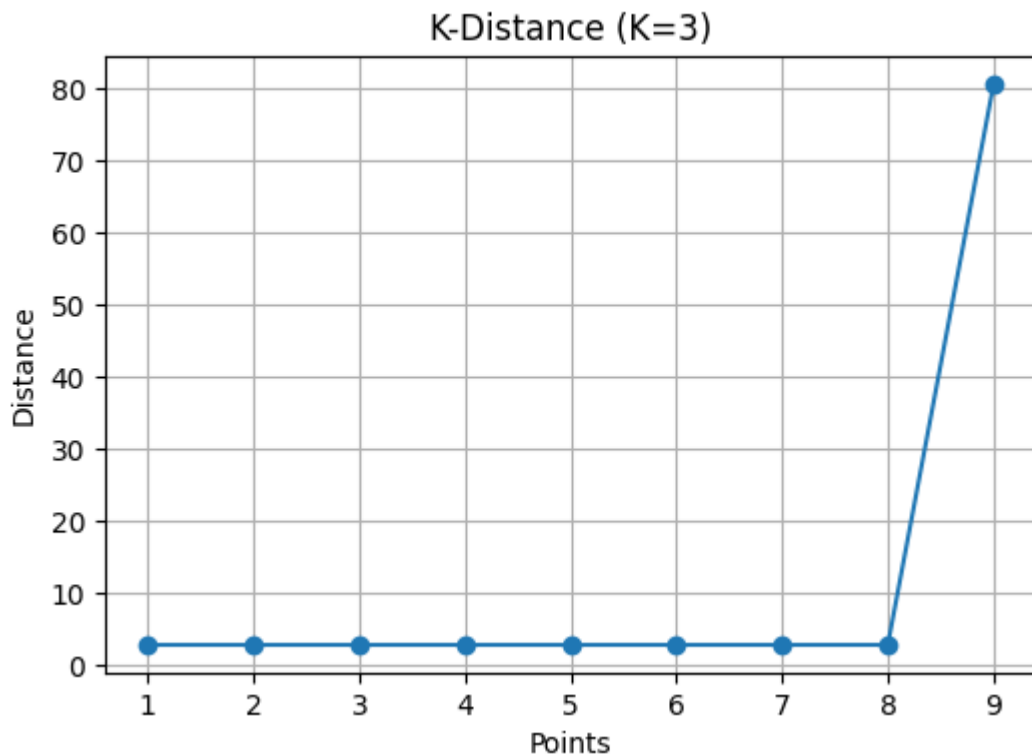


Figure 8. Biểu đồ K-Distance với $k = 3$

Kết luận: Biểu đồ trên cho thấy khoảng cách duy trì ổn định từ điểm 1 đến điểm 8 đến khi bắt đầu tăng đột biến tại điểm thứ 9, tức là có một "góc gấp" (elbow) với k-distance khoảng 2.828. Vì vậy, ta làm tròn lên để đảm bảo bao phủ cụm và $Eps = 3$.

3. Phân loại Core Point, Border Point, Noise

Với $Eps = 3$, ta duyệt từng điểm để chọn các điểm có khoảng cách nhỏ hơn hoặc bằng Eps làm điểm lân cận. Do $MinPts = 4$, như vậy, nếu một điểm có tối thiểu 3 điểm lân cận (không tính điểm đang xét) từ Bảng 3.2 thì đó là Điểm lõi (Core Point).

Bước 1: Chọn điểm bất kỳ chưa được phân cụm để xét

Chọn C1: Xung quanh C1 có 3 điểm lân cận (C2, C3, C4) trong bán kính $Eps = 3$ dẫn đến hệ quả là đạt điều kiện Core Point. Từ đó hình thành Cụm 1 (Nhóm Khách hàng Tiết kiệm).

Xét C1			
Điểm lân cận	Số điểm lân cận	Loại điểm	Cụm
C2	3	Core Point	1
C3	3	Core Point	1
C4	3	Core Point	1

Figure 9. Bảng xét điểm lân cận xung quanh C1

Bước 2: Mở rộng cụm C1 với các điểm Core Point lân cận

Vì C2, C3, C4 đều là Core Point nên thuật toán tiếp tục mở rộng. Tuy nhiên, ngoài các điểm trong nội bộ Cụm 1, không còn điểm nào khác nằm trong bán kính $Eps = 3$ của chúng. Quá trình mở rộng Cụm 1 kết thúc.

Kết luận: Cụm 1 bao gồm các điểm C1, C2, C3, C4.

Bước 3: Xét điểm chưa gắn nhãn tiếp theo

Chọn C5: Xung quanh C5 có 3 điểm lân cận (C6, C7, C8) trong bán kính $Eps = 3$ dẫn đến hệ quả là đạt điều kiện Core Point. Từ đó hình thành Cụm 2 (Nhóm Khách hàng VIP).

Xét C5			
Điểm lân cận	Số điểm lân cận	Loại điểm	Cụm
C6	3	Core Point	2
C7	3	Core Point	2
C8	3	Core Point	2

Figure 10. Bảng xét điểm lân cận xung quanh C5

Quá trình mở rộng Cụm 2 diễn ra tương tự Cụm 1 và dừng lại vì không còn điểm lân cận mới nào.

Kết luận: Cụm 2 bao gồm các điểm C5, C6, C7, C8.

Bước 4: Xét điểm chưa duyệt còn lại

Chọn O1: Khoảng cách từ O1 đến điểm gần nhất là 78.638 (vượt quá xa Eps = 3). O1 không có bất kỳ điểm lân cận nào. Từ đó xác định O1 là Điểm nhiễu (Noise).

Như vậy, kết quả phân loại cuối cùng là:

Điểm lõi (Core Point)	Điểm biên (Border Point)	Điểm nhiễu (Noise)
C1,C2,C3,C4,C5,C6,C7, C8	Không có (do cấu trúc dữ liệu minh họa hoàn hảo)	O1

Figure 11. Bảng phân cụm cuối cùng cho bài toán phân cụm khách hàng

4. Thêm điểm mới và cập nhật cụm (Incremental)

Để minh họa cơ chế cập nhật tăng cường (Incremental) – ưu điểm vượt trội giúp tiết kiệm tài nguyên tính toán so với DBSCAN truyền thống – đồ án tiến hành giả lập thêm một điểm dữ liệu mới vào hệ thống: C9 (20, 24).

Các tham số môi trường giữ nguyên: Eps = 3, MinPts = 4.

Bước 1: Truy vấn lân cận cục bộ (Local Range Query)

Thay vì tính lại toàn bộ ma trận khoảng cách 10 x 10, thuật toán chỉ tính toán khoảng cách từ điểm mới C9 đến các điểm hiện có để tìm láng giềng trong bán kính Eps = 3:

Cặp điểm xét	Tính khoảng cách	So sánh với Eps = 3
$d(C9, C1)$	$\sqrt{(20 - 20)^2 + (24 - 20)^2} = 4$	Loại

$d(C9,C2)$	$\sqrt{(20-22)^2 + (24-22)^2} = 2.828$	Hợp lệ
$d(C9,C3)$	$\sqrt{(22-20)^2 + (24-20)^2} = 4.472$	Loại
$d(C9,C4)$	$\sqrt{(20-20)^2 + (24-22)^2} = 2$	Hợp lệ

Figure 12. Tiến hành phân cụm xét theo điểm mới C9

Chú ý: Các điểm C5, C6, C7, C8, O1 ở khoảng cách >80 nên bị loại ngay lập tức.

=> Kết quả: Tập lân cận của C9 là {C4, C2}.

Bước 2: Phân loại điểm mới C9

Kiểm tra Core Point: Điểm C9 có 2 láng giềng. Vì $2 < \text{MinPts}$, C9 không phải là Điểm lõi (Core Point).

Kiểm tra Border Point: C9 nằm trong vùng lân cận của C4 và C2

Kết luận: C9 được gán nhãn là Điểm biên (Border Point).

5. Kết quả phân cụm

Sau khi tiến hành phân cụm thì ta nhận được kết quả bao gồm 2 cụm và 1 điểm nhiễu:

Cụm C1 (Nhóm Tiết kiệm): C1, C2, C3, C4, C9.

- Trong đó C1, C2, C3, C4 là điểm lõi.
- Các điểm biên là C9.

Cụm C2 (Nhóm VIP): C5, C6, C7, C8.

- Trong đó C5, C6, C7, C8 là điểm lõi.
- Các điểm biên là: Không có.

Điểm nhiễu (Noise): O1.

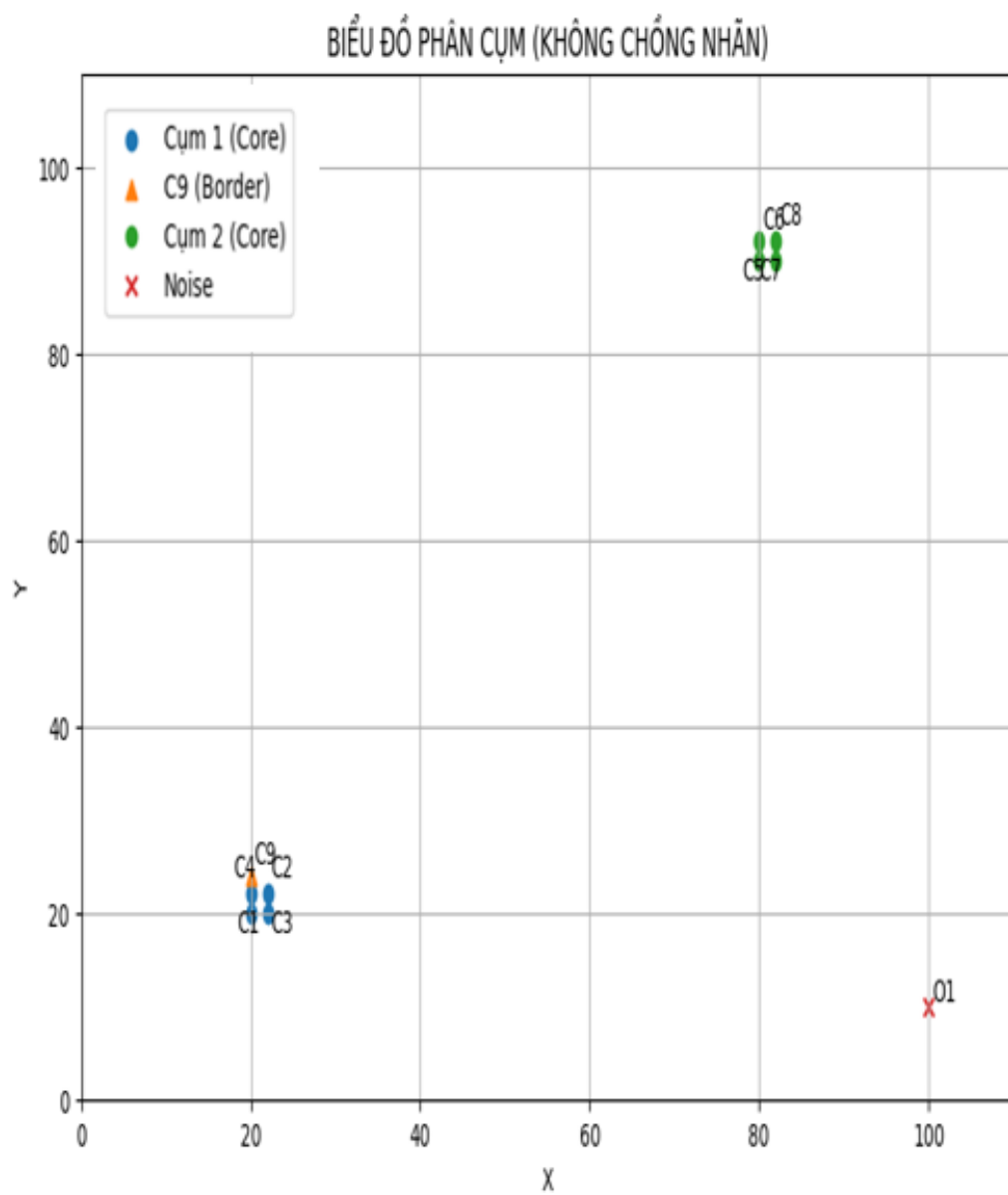


Figure 13. Kết quả phân cụm dựa theo bài toán phân cụm khách hàng

CHƯƠNG 4: TRIỂN KHAI INCREMENTAL DBSCAN

I. Giới thiệu bộ dữ liệu

Trong đồ án này, bộ dữ liệu được sử dụng là dữ liệu phi tuyến tính nhằm kiểm tra khả năng phân cụm của các thuật toán. Cụ thể, dữ liệu được tạo bằng hàm `make_moons` trong thư viện Scikit-learn.

Đây là một dạng dữ liệu kinh điển với hai cụm có hình dạng cong, lồng vào nhau, không thể tách tuyến tính.

Các đặc điểm chính của dataset:

- Số lượng điểm dữ liệu: 15,000 điểm
- Số chiều: 2 chiều (x, y)
- Có nhiễu: $\text{noise} = 0.08$
- Phân bố: phi tuyến tính (non-linear)

Ý nghĩa lựa chọn dataset:

- Phù hợp để đánh giá DBSCAN và Incremental DBSCAN
- Làm nổi bật hạn chế của K-Means (không xử lý được cụm cong)
- Phù hợp với bài toán thực tế có cấu trúc dữ liệu phức tạp

II. Giới thiệu bộ dữ liệu

Sau khi dữ liệu được tạo ra, bước tiền xử lý được thực hiện nhằm đảm bảo chất lượng đầu vào cho các thuật toán phân cụm. Mặc dù dữ liệu ban đầu không chứa giá trị thiếu, quy trình xử lý vẫn được thực hiện đầy đủ để đảm bảo tính tổng quát.

Quá trình tiền xử lý bao gồm các bước chính sau:

- Trích xuất hai đặc trưng chính là x và y từ tập dữ liệu
- Xử lý giá trị thiếu bằng cách thay thế bằng giá trị trung vị
- Chuẩn hóa dữ liệu bằng StandardScaler

Việc chuẩn hóa đóng vai trò đặc biệt quan trọng, bởi các thuật toán như DBSCAN sử dụng khoảng cách Euclidean để xác định vùng lân cận. Nếu dữ liệu không được đưa về cùng thang đo, kết quả phân cụm có thể bị sai lệch đáng kể.

Sau khi hoàn tất, dữ liệu được lưu dưới dạng file CSV và được trực quan hóa bằng biểu đồ phân tán, qua đó có thể quan sát rõ cấu trúc hai cụm cong đặc trưng.

III. Cài đặt Incremental DBSCAN bằng Python

```
class IncrementalDBSCAN:
    def __init__(self, eps=0.15, min_samples=20):
        self.eps = eps
        self.min_samples = min_samples
        self.data = []
        self.labels = []
        self.cluster_id = 0
        self.tree = None
```

Figure 14. Minh họa cấu trúc khởi tạo của thuật toán Incremental DBSCAN

Trong đồ án này, thuật toán Incremental DBSCAN được cài đặt thủ công dưới dạng một lớp trong Python nhằm làm rõ cơ chế hoạt động nội tại thay vì sử dụng trực tiếp từ thư viện. Như thể hiện trong Hình 4.3.1, lớp **IncrementalDBSCAN** được thiết kế với các tham số quan trọng gồm **eps** và **min_samples**, tương ứng với bán kính lân cận và ngưỡng mật độ tối thiểu.

Bên cạnh đó, cấu trúc dữ liệu nội bộ của thuật toán được xây dựng bao gồm:

- **self.data**: lưu trữ toàn bộ điểm dữ liệu đã được đưa vào hệ thống
- **self.labels**: lưu nhãn cụm tương ứng của từng điểm
- **self.cluster_id**: biến quản lý số lượng cụm
- **self.tree**: cấu trúc KD-Tree phục vụ truy vấn

Một điểm đáng chú ý là việc sử dụng giá trị đặc biệt -99 để đánh dấu các điểm chưa được xử lý, giúp thuật toán dễ dàng kiểm soát trạng thái của dữ liệu trong quá trình cập nhật.

```
def build_tree(self):  
    if len(self.data) > 0:  
        self.tree = KDTree(np.array(self.data))
```

Figure 15. Minh họa việc xây dựng KD-Tree từ dữ liệu hiện tại

Để tối ưu hóa quá trình tìm kiếm lân cận, nhóm triển khai sử dụng thuật toán KD-Tree. Cụ thể là sau mỗi lần thêm dữ liệu mới, KD-Tree được xây dựng lại dựa trên toàn bộ tập dữ liệu hiện tại.

Cách tiếp cận này mang lại hiệu quả rõ rệt:

- Giảm độ phức tạp khi truy vấn vùng lân cận
- Tránh việc tính toán khoảng cách với toàn bộ điểm dữ liệu
- Phù hợp với dữ liệu hai chiều như trong bài toán moons

```
def region_query(self, point_idx):  
    return self.tree.query_radius([self.data[point_idx]], r=self.eps)[0]
```

Figure 16. Minh họa truy vấn các điểm trong bán kính eps

Hàm `region_query` sử dụng phương thức `query_radius` của KD-Tree để tìm tất cả các điểm nằm trong bán kính `eps`. Đây là bước cốt lõi của thuật toán, bởi số lượng điểm trong vùng lân cận sẽ quyết định việc phân loại điểm thành lõi, biên hay nhiễu.

Việc sử dụng KD-Tree giúp:

- Tăng tốc đáng kể so với duyệt tuần tự
- Đảm bảo hiệu suất khi dữ liệu tăng dần theo thời gian

```

for i in range(0, len(X_coords), BATCH_SIZE):
    batch = X_coords[i : i + BATCH_SIZE]
    start = time.time()
    inc_model.partial_fit(batch)
    inc_batch_times.append(time.time() - start)

```

Figure 17. Minh họa quá trình đưa dữ liệu vào hệ thống theo từng batch

Khác với DBSCAN truyền thống, toàn bộ dữ liệu trong bài toán không được xử lý cùng một lúc mà được chia thành các batch nhỏ với kích thước cố định và lần lượt đưa vào hệ thống thông qua hàm `partial_fit`. Quá trình này mô phỏng sát với các hệ thống dữ liệu thực tế, nơi dữ liệu thường được sinh ra liên tục theo thời gian thay vì tồn tại đầy đủ ngay từ đầu.

Trong mỗi lần gọi `partial_fit`, các điểm dữ liệu mới sẽ được thêm vào danh sách hiện tại và được gán trạng thái chưa xử lý. Sau đó, thuật toán tiến hành xây dựng lại KD-Tree và bắt đầu kiểm tra từng điểm mới để xác định vai trò của chúng trong cấu trúc cụm. Cách tiếp cận này giúp tránh việc phải chạy lại toàn bộ thuật toán từ đầu, từ đó tiết kiệm đáng kể chi phí tính toán khi dữ liệu tăng lên.

```

def expand_cluster(self, start_idx, neighbors, target_id):
    old_label = self.labels[start_idx]
    if old_label > 0 and old_label != target_id:
        self.labels = [target_id if l == old_label else l for l in self.labels]

    self.labels[start_idx] = target_id
    queue = set(neighbors)

    while len(queue) > 0:
        current_idx = queue.pop()
        current_label = self.labels[current_idx]

        if current_label > 0 and current_label != target_id:
            self.labels = [target_id if l == current_label else l for l in self.labels]

        if self.labels[current_idx] <= 0:
            is_unvisited = (self.labels[current_idx] == -99)
            self.labels[current_idx] = target_id

            if is_unvisited:
                new_neighbors = self.region_query(current_idx)
                if len(new_neighbors) >= self.min_samples:
                    for n in new_neighbors:
                        queue.add(n)

```

Figure 18. Minh họa quá trình mở rộng cụm từ một điểm lõi

Khi một điểm được xác định là điểm lõi, thuật toán sẽ thực hiện quá trình mở rộng cụm thông qua hàm `expand_cluster`. Thuật toán sử dụng một cấu trúc dạng hàng đợi để lan rộng cụm đến các điểm có khả năng tiếp cận mật độ. Mỗi điểm trong vùng lân cận sẽ được kiểm tra, nếu bản thân nó cũng là điểm lõi thì vùng lân cận của nó tiếp tục được thêm vào quá trình mở rộng.

Cơ chế này đảm bảo rằng toàn bộ các điểm có liên kết mật độ với nhau sẽ được gom về cùng một cụm, từ đó hình thành nên các cụm có hình dạng bất kỳ thay vì bị giới hạn trong các dạng hình học đơn giản.

```
while len(queue) > 0:
    current_idx = queue.pop()
    current_label = self.labels[current_idx]

    if current_label > 0 and current_label != target_id:
        self.labels = [target_id if l == current_label else l for l in self.labels]

    if self.labels[current_idx] <= 0:
        is_unvisited = (self.labels[current_idx] == -99)
        self.labels[current_idx] = target_id

    if is_unvisited:
        new_neighbors = self.region_query(current_idx)
        if len(new_neighbors) >= self.min_samples:
            for n in new_neighbors:
                queue.add(n)
```

Figure 19. Minh họa cơ chế hợp nhất cụm khi một điểm kết nối nhiều cụm

Một trong những điểm quan trọng nhất trong phần cài đặt là cơ chế hợp nhất cụm, thể hiện rõ khi một điểm dữ liệu mới có vùng lân cận chứa các điểm thuộc nhiều cụm khác nhau. Thuật toán không tạo ra một cụm mới mà thực hiện việc đồng nhất nhãn giữa các cụm này bằng cách cập nhật lại toàn bộ các nhãn liên quan về cùng một `cluster_id`. Khi đó, điểm mới đóng vai trò như một cầu nối mật độ, làm lộ ra sự liên thông giữa các cụm trước đó.

Cách xử lý này giúp đảm bảo rằng cấu trúc phân cụm không bị phân mảnh và vẫn phản ánh đúng bản chất kết nối mật độ của dữ liệu. Đây cũng là yếu tố giúp Incremental DBSCAN duy trì được chất lượng phân cụm tương đương DBSCAN truyền thống trong khi vẫn tối ưu về mặt hiệu năng.

```
def get_labels(self):
    return np.array(self.labels)
```

Figure 20. Minh họa việc gán nhãn phân cụm vào tập dữ liệu

Sau khi hoàn tất quá trình cập nhật, toàn bộ nhãn cụm được trích xuất thông qua hàm `get_labels` và gán vào DataFrame chính để phục vụ cho các bước trực quan hóa và đánh giá. Kết quả này là đầu ra cuối cùng của thuật toán và là cơ sở để so sánh với các phương pháp khác trong các phần tiếp theo.

IV. Biểu diễn kết quả

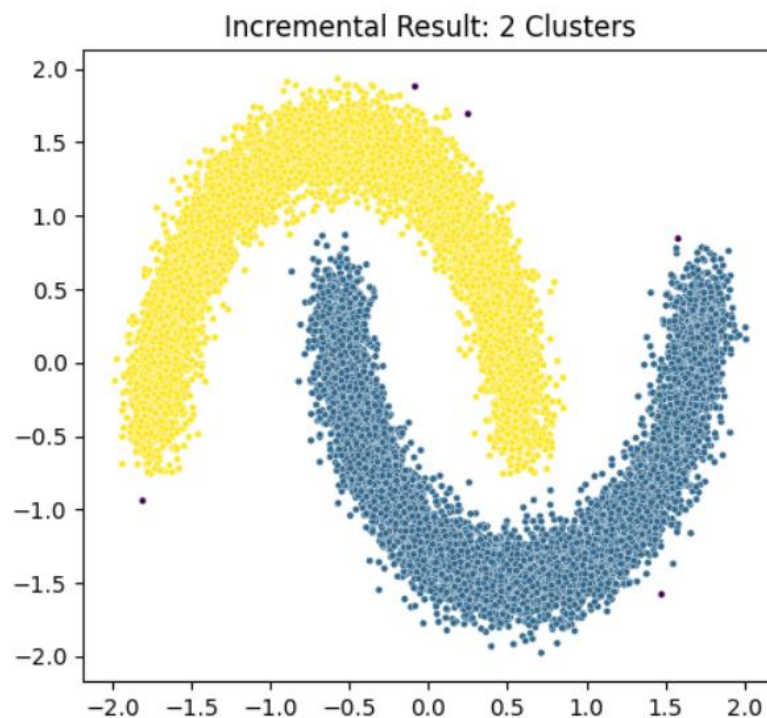


Figure 21. Minh họa kết quả phân cụm của Incremental DBSCAN trên dữ liệu moons

Sau khi hoàn tất quá trình phân cụm, kết quả được trực quan hóa thông qua biểu đồ phân tán trong không gian hai chiều. Mỗi cụm được biểu diễn bằng một màu sắc khác nhau, trong khi các điểm nhiễu được gán nhãn riêng biệt, từ đó giúp người quan sát dễ dàng nhận diện cấu trúc dữ liệu.

Quan sát từ hình có thể thấy rằng thuật toán Incremental DBSCAN đã tái hiện được rõ ràng hai cụm có dạng cong đặc trưng của dữ liệu moons. Đây là một điểm quan trọng bởi dạng dữ liệu này vốn không phù hợp với các thuật toán phân cụm dựa trên khoảng cách như K-Means. Việc các cụm được phát hiện liên tục và không bị đứt đoạn cho thấy cơ chế mở rộng cụm và kết nối mật độ trong quá trình incremental vẫn được duy trì chính xác.

Một điểm đáng chú ý là mặc dù dữ liệu được đưa vào theo từng batch nhỏ, cấu trúc cụm cuối cùng không bị sai lệch so với cách xử lý toàn bộ dữ liệu. Điều này cho thấy thuật toán không chỉ tối ưu về mặt hiệu năng mà còn đảm bảo tính ổn định về chất lượng phân cụm.

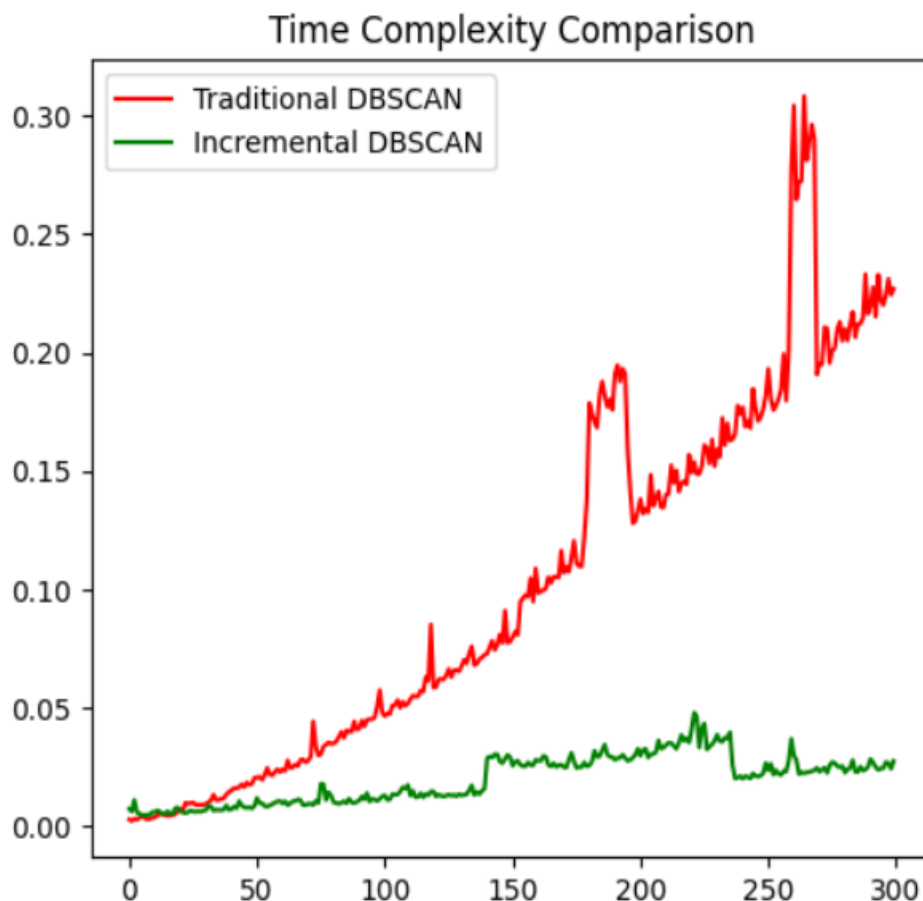


Figure 22. So sánh thời gian xử lý giữa DBSCAN và Incremental DBSCAN

Bên cạnh việc đánh giá kết quả phân cụm, đề án còn trực quan hóa sự khác biệt về thời gian xử lý giữa hai phương pháp. Biểu đồ này được xây dựng dựa trên thời gian xử lý của từng batch dữ liệu.

Từ hình vẽ có thể nhận thấy rằng DBSCAN truyền thống có xu hướng gia tăng thời gian xử lý theo từng batch, do mỗi lần cập nhật đều phải thực hiện phân cụm lại trên toàn bộ dữ liệu đã tích lũy. Ngược lại, Incremental DBSCAN chỉ xử lý phần dữ liệu mới và cập nhật cục bộ, do đó thời gian xử lý giữa các batch tương đối ổn định.

Sự khác biệt này phản ánh rõ bản chất của hai phương pháp, trong đó Incremental DBSCAN tận dụng thông tin đã có để tránh tính toán dư thừa, từ đó mang lại hiệu quả cao hơn trong các bài toán dữ liệu lớn hoặc dữ liệu thay đổi liên tục.

V. Đánh giá hiệu suất

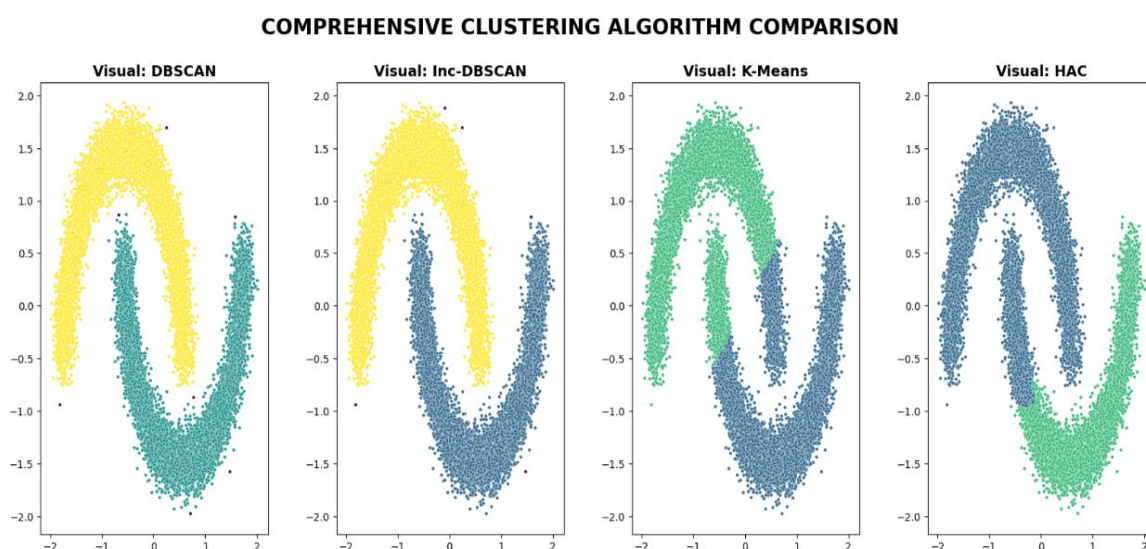


Figure 23. Minh họa kết quả phân cụm của các thuật toán khác nhau

Để đánh giá một cách toàn diện hiệu quả của các thuật toán, đề án sử dụng kết hợp giữa trực quan hóa và các chỉ số định lượng, như thể hiện trong Hình 4.5.1. Hình này tổng hợp đồng thời kết quả phân cụm của các thuật toán cùng với các chỉ số đánh giá như Silhouette Score, Davies-Bouldin Index và Calinski-Harabasz Index, từ đó cung cấp một cái nhìn tổng thể về chất lượng phân cụm.

Xét về mặt trực quan, có thể nhận thấy rằng DBSCAN và Incremental DBSCAN thể hiện rõ khả năng phát hiện các cụm có hình dạng phi tuyến, đặc biệt phù hợp với cấu trúc dữ liệu dạng moons. Các cụm được hình thành liên tục và ít bị nhiễu ảnh hưởng, trong khi các điểm ngoại lai được tách biệt rõ ràng. Ngược lại, các thuật toán như K-Means và HAC có xu hướng phân chia dữ liệu theo dạng hình học đơn giản, dẫn đến việc không phản ánh đúng cấu trúc cong của dữ liệu.

Về mặt định lượng, các chỉ số đánh giá cho thấy Incremental DBSCAN đạt kết quả tương đương với DBSCAN truyền thống. Điều này cho thấy rằng việc cập nhật dữ liệu theo từng batch không làm suy giảm chất lượng phân cụm, mà vẫn giữ được tính chính xác trong việc xác định cấu trúc cụm. Tuy nhiên, cần lưu ý rằng với các dữ liệu có hình dạng phức tạp, một số chỉ số như Silhouette Score có thể không đạt giá trị cao dù kết quả phân cụm là hợp lý, do khoảng cách giữa các cụm không đồng đều.

Xét tổng thể, điểm khác biệt quan trọng nhất nằm ở hiệu năng xử lý. Incremental DBSCAN cho thấy khả năng duy trì thời gian xử lý ổn định nhờ cơ chế cập nhật cục bộ, trong khi DBSCAN truyền thống phải thực hiện lại toàn bộ quá trình khi dữ liệu thay đổi. Điều này giúp Incremental DBSCAN trở thành lựa chọn phù hợp hơn trong các bài toán thực tế, nơi dữ liệu có xu hướng tăng dần theo thời gian.

CHƯƠNG 5: SO SÁNH INCREMENTAL DBSCAN VỚI CÁC THUẬT TOÁN PHÂN CỤM KHÁC

I. Giới thiệu chung

Trong lĩnh vực khai phá dữ liệu (Data Mining), bài toán phân cụm đóng vai trò quan trọng trong việc khám phá cấu trúc tiềm ẩn của dữ liệu mà không cần nhãn trước. Có nhiều thuật toán phân cụm đã được đề xuất, mỗi thuật toán đều có ưu điểm và hạn chế riêng, phụ thuộc vào đặc điểm dữ liệu và mục tiêu sử dụng.

Trong chương này, thuật toán **Incremental DBSCAN (Inc-DBSCAN)** được đánh giá và so sánh với ba thuật toán phổ biến gồm **K-Means**, **Hierarchical Agglomerative Clustering (HAC)** và **DBSCAN truyền thống**. Mục tiêu là làm rõ hiệu quả của Inc-DBSCAN trong các tình huống dữ liệu thực tế, đặc biệt là khi dữ liệu có tính động (liên tục thay đổi theo thời gian).

Việc so sánh được thực hiện dựa trên cả hai khía cạnh:

- **Định lượng:** thông qua các chỉ số đánh giá như Silhouette Score, Davies-Bouldin Index và Calinski-Harabasz Score.
- **Định tính:** dựa trên khả năng xử lý nhiễu, tốc độ cập nhật dữ liệu và độ phức tạp tính toán.

II. Các thuật toán được sử dụng trong so sánh

1. K-Means Clustering

K-Means là một trong những thuật toán phân cụm phổ biến nhất, hoạt động dựa trên nguyên lý chia dữ liệu thành k cụm sao cho tổng bình phương khoảng cách từ các điểm đến tâm cụm là nhỏ nhất. Thuật toán này có ưu điểm là đơn giản, dễ triển khai và có tốc độ xử lý nhanh.

Tuy nhiên, K-Means có một số hạn chế đáng kể:

- Cần xác định trước số cụm k
- Nhạy cảm với nhiễu và điểm ngoại lai
- Không phù hợp với dữ liệu có hình dạng phức tạp (phi tuyến)

2. Hierarchical Agglomerative Clustering (HAC)

HAC là phương pháp phân cụm phân cấp, bắt đầu từ việc xem mỗi điểm dữ liệu là một cụm riêng biệt, sau đó liên tục gộp các cụm gần nhau nhất cho đến khi đạt điều kiện dừng.

Ưu điểm:

- Không cần xác định số cụm ban đầu
- Có thể biểu diễn cấu trúc dữ liệu dưới dạng cây phân cấp

Nhược điểm:

- Độ phức tạp tính toán cao
- Không phù hợp với tập dữ liệu lớn
- Không xử lý tốt dữ liệu nhiễu

3. DBSCAN truyền thống

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) là thuật toán phân cụm dựa trên mật độ, có khả năng phát hiện các cụm có hình dạng bất kỳ và loại bỏ nhiễu.

Ưu điểm:

- Không cần xác định số cụm trước
- Phát hiện tốt các cụm phi tuyến
- Xử lý nhiễu hiệu quả

Nhược điểm:

- Nhạy cảm với tham số (ϵ , minPts)
- Không phù hợp với dữ liệu có mật độ không đồng đều

- Không hỗ trợ cập nhật dữ liệu động

4. Incremental DBSCAN

Inc-DBSCAN là phiên bản cải tiến của DBSCAN, được thiết kế để xử lý dữ liệu động. Thay vì phải chạy lại toàn bộ thuật toán khi có dữ liệu mới, Inc-DBSCAN chỉ cập nhật các cụm bị ảnh hưởng bởi dữ liệu mới.

Ưu điểm nổi bật:

- Hỗ trợ cập nhật dữ liệu theo thời gian thực
- Giữ được khả năng phát hiện cụm phi tuyến
- Giảm chi phí tính toán khi dữ liệu thay đổi

III. Phân tích chi tiết các tiêu chí đánh giá

1. Khả năng xử lý dữ liệu luôn thay đổi (Dynamic Data Handling)

Trong các hệ thống thực tế như giám sát mạng, dữ liệu thường không cố định mà liên tục được cập nhật theo thời gian. Do đó, một thuật toán phân cụm hiệu quả cần có khả năng thích nghi với sự thay đổi này.

K-Means, HAC và DBSCAN truyền thống đều là các thuật toán “tĩnh”, nghĩa là khi có dữ liệu mới, toàn bộ quá trình phân cụm phải được thực hiện lại từ đầu. Điều này dẫn đến chi phí tính toán cao và không phù hợp với hệ thống thời gian thực.

Incremental DBSCAN (Inc-DBSCAN) khắc phục hạn chế này bằng cách chỉ cập nhật các cụm bị ảnh hưởng bởi dữ liệu mới. Cụ thể, thuật toán sẽ kiểm tra vùng lân cận của điểm mới và điều chỉnh cấu trúc cụm tương ứng mà không cần xử lý lại toàn bộ tập dữ liệu.

Cơ chế cập nhật cục bộ của Inc-DBSCAN giúp giảm đáng kể độ phức tạp tính toán từ $O(n)$ xuống gần $O(k)$ với k là số điểm bị ảnh hưởng (thường rất nhỏ so với toàn bộ dữ liệu). Điều này đặc biệt quan trọng trong các hệ thống lớn, nơi dữ liệu có thể lên đến hàng triệu điểm.

Kết luận tiêu chí: Inc-DBSCAN là thuật toán duy nhất trong nhóm có khả năng xử lý dữ liệu động một cách hiệu quả, tạo ra lợi thế lớn trong các ứng dụng thực tế.

2. Hiệu suất phân cụm (Clustering Quality)

Hiệu suất phân cụm được đánh giá thông qua ba chỉ số quan trọng:

a. Silhouette Score

Chỉ số Silhouette đo lường mức độ tương đồng của một điểm dữ liệu với cụm của nó so với các cụm khác. Giá trị nằm trong khoảng $[-1, 1]$, trong đó:

- Gần 1: phân cụm tốt, các cụm tách biệt rõ ràng
- Gần 0: các cụm chồng lấn
- Âm: phân cụm sai

Phân tích kết quả:

- K-Means đạt giá trị cao nhất (~ 0.495) $\rightarrow$ cho thấy các cụm được tách biệt tốt về mặt khoảng cách.
- HAC (~ 0.452) cũng cho kết quả tốt, phản ánh khả năng gom cụm hợp lý.
- Inc-DBSCAN (~ 0.34) cao hơn DBSCAN (~ 0.316), cho thấy việc cải tiến đã giúp tăng độ rõ ràng của cụm.

Nhận xét: Mặc dù K-Means có Silhouette cao nhất, nhưng điều này chủ yếu do dữ liệu được “ép” vào dạng cụm hình cầu, không phản ánh đúng bản chất cấu trúc phi tuyến của dữ liệu.

b. Davies-Bouldin Index (DBI)

DBI đo lường mức độ phân tán trong cụm và khoảng cách giữa các cụm. Giá trị càng thấp càng tốt.

Phân tích kết quả:

- K-Means (~ 0.81) và HAC (~ 0.845) có giá trị thấp $\rightarrow$ cụm chặt và ít chồng lấn
- Inc-DBSCAN (~ 2.75) tốt hơn DBSCAN (~ 3.84) $\rightarrow$ cải thiện rõ rệt

DBSCAN truyền thống có xu hướng tạo nhiều cụm nhỏ hoặc cụm không đồng đều, dẫn đến khoảng cách giữa các cụm không rõ ràng, làm tăng DBI. Inc-DBSCAN cải thiện điều này bằng cách cập nhật cấu trúc cụm linh hoạt hơn.

c. *Calinski-Harabasz Score (CH Score)*

Chỉ số này đánh giá tỷ lệ giữa phương sai giữa các cụm và phương sai trong cụm. Giá trị càng cao càng tốt.

Phân tích kết quả:

- K-Means (~20797) cao nhất → cụm phân tách rõ
- HAC (~16606) đứng thứ hai
- DBSCAN và Inc-DBSCAN (~6488) thấp hơn đáng kể

Nhận xét: Chỉ số CH thiên về các cụm có dạng hình học đơn giản, do đó K-Means có lợi thế. Tuy nhiên, với dữ liệu phi tuyến, chỉ số này không phản ánh đầy đủ chất lượng phân cụm.

Kết luận tiêu chí hiệu suất:

- K-Means tối ưu về chỉ số nhưng hạn chế về hình dạng cụm
- Inc-DBSCAN cải thiện đáng kể so với DBSCAN
- Các chỉ số đánh giá cần được xem xét cùng với đặc điểm dữ liệu

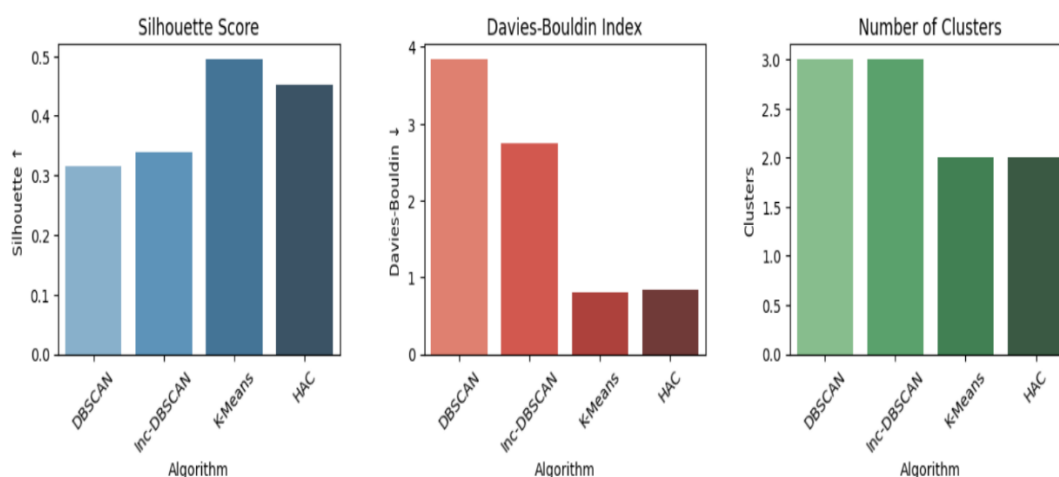


Figure 24. Minh họa hiệu suất

3. Khả năng xử lý dữ liệu nhiễu (Noise Handling)

Trong dữ liệu thực tế, sự xuất hiện của nhiễu (outliers) là không thể tránh khỏi. Một thuật toán phân cụm hiệu quả cần có khả năng nhận diện và loại bỏ các điểm này.

DBSCAN và Inc-DBSCAN: sử dụng khái niệm mật độ để xác định điểm lõi, điểm biên và điểm nhiễu. Các điểm không thuộc cụm nào sẽ được gán nhãn riêng (thường là -1).

K-Means và HAC: không có cơ chế xử lý nhiễu, tất cả các điểm đều bị ép vào một cụm nào đó.

Việc ép các điểm nhiễu vào cụm (trong K-Means và HAC) làm giảm chất lượng phân cụm, đặc biệt trong các bài toán có dữ liệu không sạch. 2 thuật toán DBSCAN và Inc-DBSCAN vượt trội trong việc xử lý dữ liệu nhiễu, giúp tăng độ tin cậy của kết quả.

4. Độ phức tạp tính toán (Computational Complexity)

Độ phức tạp ảnh hưởng trực tiếp đến khả năng mở rộng của thuật toán.

- **K-Means:** $O(nkt)$ → nhanh, phù hợp dữ liệu lớn
- **HAC:** $O(n^2 \log n)$ → rất chậm với dữ liệu lớn
- **DBSCAN:** $O(n \log n)$ → trung bình
- **Inc-DBSCAN:** giảm đáng kể khi cập nhật

Trong môi trường dữ liệu lớn, HAC gần như không khả thi. Inc-DBSCAN tận dụng kết quả trước đó để giảm chi phí tính toán, đặc biệt khi số lượng dữ liệu mới nhỏ.

5. Thời gian cập nhật dữ liệu (Update Time)

Đây là tiêu chí quan trọng trong các hệ thống thời gian thực.

- **Inc-DBSCAN:** chỉ cập nhật cục bộ → thời gian gần như tuyến tính theo số điểm mới
- **Các thuật toán khác:** phải xử lý lại toàn bộ → thời gian tăng theo kích thước dữ liệu

Phân tích: Khi dữ liệu tăng dần theo thời gian, sự khác biệt về hiệu năng giữa Inc-DBSCAN và các thuật toán khác sẽ ngày càng lớn.

IV. Phân tích kết quả thực nghiệm

Dựa trên kết quả trực quan hóa và bảng số liệu:

- **K-Means** cho kết quả tốt nhất về mặt chỉ số, tuy nhiên không nhận diện đúng cấu trúc dữ liệu phi tuyến.
- **HAC** cho kết quả ổn định nhưng chi phí tính toán cao.
- **DBSCAN** nhận diện tốt hình dạng cụm nhưng hiệu suất chưa cao.
- **Inc-DBSCAN** cải thiện đáng kể so với DBSCAN, đặc biệt về chỉ số Davies-Bouldin.

Ngoài ra, Inc-DBSCAN vẫn giữ được khả năng phát hiện cụm có hình dạng phức tạp, điều mà K-Means không làm được.

Kết quả thực nghiệm cho thấy một sự đánh đổi rõ rệt: K-Means tối ưu về tốc độ và chỉ số hình học nhưng thất bại trong việc nắm bắt bản chất dữ liệu phi tuyến. Ngược lại, Incremental DBSCAN chấp nhận một mức chỉ số định lượng vừa phải để đổi lấy khả năng nhận diện cấu trúc phức tạp và khả năng thích nghi với dữ liệu động, yếu tố then chốt trong các ứng dụng thực tế hiện nay.

V. Kết luận

Qua quá trình so sánh, có thể rút ra một số nhận định quan trọng:

- Không có thuật toán nào là tối ưu trong mọi trường hợp.
- K-Means phù hợp với dữ liệu đơn giản và yêu cầu tốc độ cao.
- HAC phù hợp với dữ liệu nhỏ và cần phân tích cấu trúc phân cấp.
- DBSCAN phù hợp với dữ liệu có nhiễu và cấu trúc phức tạp.

Inc-DBSCAN là lựa chọn tối ưu cho dữ liệu động, nơi yêu cầu cập nhật nhanh và xử lý nhiễu hiệu quả.

Tiêu chí so sánh	K-Means	HAC (Phân cấp)	DBSCAN (Tĩnh)	Incremental DBSCAN
Nguyên lý chính	Phân hoạch (Tâm cụm)	Phân cấp (Gộp cụm)	Mật độ (Density)	Mật độ (Cập nhật cục bộ)
Số cụm (K)	Phải xác định trước	Tự động/Cắt cây	Tự động xác định	Tự động xác định
Hình dạng cụm	Chỉ hình cầu/lõi	Đa dạng	Bất kỳ (Phi tuyến)	Bất kỳ (Phi tuyến)
Xử lý nhiễu	Kém (Ép vào cụm)	Kém	Rất tốt	Rất tốt
Dữ liệu động	Không (Phải chạy lại)	Không (Phải chạy lại)	Không (Phải chạy lại)	Hỗ trợ cập nhật Real-time
Độ phức tạp	$O(nkt)$ (Nhanh)	$O(n^2 \log n)$ (Chậm)	$O(n \log n)$ (Trung bình)	$O(k)$ (Với k là vùng ảnh hưởng)
Silhouette Score	Cao nhất (~0.49)	Khá (~0.45)	Trung bình (~0.31)	Trung bình khá (~0.34)
Davies-Bouldin	Thấp (Tốt)	Thấp (Tốt)	Cao (Kém)	Cải thiện rõ rệt
Ưu điểm lớn nhất	Tốc độ rất nhanh	Cấu trúc phân cấp	Phát hiện cụm phức tạp	Tối ưu cho dữ liệu Streaming

Kết luận: Như vậy, mặc dù không đạt chỉ số cao nhất, nhưng **Incremental DBSCAN** thể hiện sự cân bằng tốt nhất giữa hiệu suất, tính linh hoạt và khả năng ứng dụng thực tế, đặc biệt trong các hệ thống dữ liệu lớn và thời gian thực.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

I. Tổng kết kết quả đạt được

Trong phạm vi đồ án này, thuật toán Incremental DBSCAN đã được nghiên cứu, xây dựng và triển khai thành công dưới dạng cài đặt thủ công bằng Python. Thay vì sử dụng trực tiếp từ thư viện, việc tự cài đặt giúp làm rõ cơ chế hoạt động nội tại của thuật toán, đặc biệt là cách xử lý dữ liệu theo hướng incremental.

Một trong những kết quả quan trọng đạt được là việc mô phỏng thành công quá trình dữ liệu đến theo từng batch, phản ánh sát với các hệ thống thực tế. Thông qua cơ chế này, thuật toán có thể cập nhật cấu trúc cụm mà không cần thực hiện lại toàn bộ quá trình phân cụm từ đầu.

Về mặt chất lượng phân cụm, kết quả thực nghiệm trên bộ dữ liệu phi tuyến (moons) cho thấy Incremental DBSCAN vẫn giữ được khả năng phát hiện cụm có hình dạng bất kỳ, tương đương với DBSCAN truyền thống. Các cụm được hình thành rõ ràng, liên tục và không bị gián đoạn, đồng thời các điểm nhiễu được nhận diện chính xác.

Về mặt hiệu năng, thuật toán thể hiện ưu thế rõ rệt so với DBSCAN truyền thống trong bối cảnh dữ liệu thay đổi liên tục. Thời gian xử lý được duy trì ổn định qua các batch nhờ cơ chế cập nhật cục bộ, từ đó giảm đáng kể chi phí tính toán. Đây là điểm khác biệt quan trọng, chứng minh tính thực tiễn của Incremental DBSCAN trong các hệ thống dữ liệu lớn.

Ngoài ra, đồ án cũng đã thực hiện so sánh toàn diện với các thuật toán khác như K-Means và HAC, qua đó làm nổi bật ưu điểm của phương pháp dựa trên mật độ trong việc xử lý dữ liệu phi tuyến và dữ liệu có nhiễu.

Tổng thể, đồ án đã đạt được mục tiêu đề ra là xây dựng một mô hình phân cụm hiệu quả, vừa đảm bảo chất lượng kết quả vừa tối ưu về hiệu năng xử lý trong môi trường dữ liệu động.

II. Hạn chế và thách thức

Mặc dù đạt được nhiều kết quả tích cực, thuật toán Incremental DBSCAN vẫn tồn tại một số hạn chế nhất định cần được xem xét.

Thứ nhất, việc lựa chọn tham số ϵ và min_samples vẫn mang tính thủ công và phụ thuộc nhiều vào đặc điểm của dữ liệu. Nếu các tham số này không được chọn phù hợp, kết quả phân cụm có thể bị sai lệch, dẫn đến việc tạo quá nhiều cụm nhỏ hoặc gộp các cụm không liên quan.

Thứ hai, trong quá trình cài đặt, cấu trúc KD-Tree được xây dựng lại sau mỗi lần cập nhật dữ liệu. Mặc dù điều này giúp tối ưu truy vấn lân cận, nhưng vẫn gây ra chi phí nhất định khi dữ liệu tăng lên. Với các tập dữ liệu cực lớn hoặc tốc độ cập nhật cao, việc tái xây dựng này có thể trở thành nút thắt hiệu năng.

Thứ ba, thuật toán hiện tại chủ yếu được thử nghiệm trên dữ liệu hai chiều. Khi mở rộng sang không gian nhiều chiều (high-dimensional data), hiệu quả của các cấu trúc như KD-Tree có thể giảm đáng kể do hiện tượng “curse of dimensionality”.

Thứ tư, cơ chế hợp nhất cụm trong Incremental DBSCAN, mặc dù hiệu quả, nhưng có thể dẫn đến chi phí cập nhật nhân lớn khi nhiều cụm bị hợp nhất cùng lúc. Điều này có thể ảnh hưởng đến hiệu năng trong các trường hợp dữ liệu có mật độ thay đổi mạnh.

Cuối cùng, việc đánh giá chất lượng phân cụm vẫn phụ thuộc nhiều vào các chỉ số truyền thống như Silhouette hay Davies-Bouldin, trong khi các chỉ số này đôi khi không phản ánh chính xác chất lượng với dữ liệu phi tuyến.

III. Đề xuất cải tiến

Từ các hạn chế đã phân tích, một số hướng cải tiến có thể được đề xuất nhằm nâng cao hiệu quả của thuật toán trong tương lai.

Một hướng quan trọng là phát triển cơ chế tự động lựa chọn tham số ϵ và min_samples , có thể dựa trên các phương pháp học máy hoặc heuristic như k-distance

graph. Điều này giúp giảm sự phụ thuộc vào người dùng và tăng tính tổng quát của thuật toán.

Bên cạnh đó, có thể nghiên cứu sử dụng các cấu trúc dữ liệu nâng cao hơn thay cho KD-Tree, chẳng hạn như Ball Tree hoặc các phương pháp approximate nearest neighbors (ANN), nhằm cải thiện hiệu năng trong không gian nhiều chiều.

Một hướng khác là tối ưu hóa cơ chế cập nhật bằng cách chỉ tái xây dựng một phần cấu trúc dữ liệu thay vì toàn bộ, từ đó giảm chi phí xử lý khi dữ liệu tăng lớn.

Ngoài ra, việc kết hợp Incremental DBSCAN với các kỹ thuật học sâu (Deep Learning) cũng là một hướng tiềm năng. Ví dụ, sử dụng autoencoder để giảm chiều dữ liệu trước khi phân cụm có thể giúp cải thiện hiệu quả trên các dữ liệu phức tạp.

Cuối cùng, có thể mở rộng thuật toán sang môi trường xử lý phân tán (distributed computing) như Apache Spark, nhằm tăng khả năng xử lý dữ liệu lớn trong thực tế.

IV. Ứng dụng thực tế

Nhờ khả năng xử lý dữ liệu động và phát hiện cụm phi tuyến, Incremental DBSCAN có tiềm năng ứng dụng rộng rãi trong nhiều lĩnh vực thực tế.

Trong lĩnh vực phân tích hành vi người dùng, thuật toán có thể được sử dụng để phân cụm người dùng theo hành vi truy cập hoặc mua sắm theo thời gian thực, từ đó hỗ trợ hệ thống gợi ý (recommendation systems).

Trong lĩnh vực an ninh mạng, Incremental DBSCAN có thể giúp phát hiện các hành vi bất thường (anomaly detection) dựa trên dữ liệu log liên tục, nhờ khả năng nhận diện nhiều hiệu quả.

Trong các hệ thống IoT và cảm biến, nơi dữ liệu được sinh ra liên tục, thuật toán có thể được sử dụng để phát hiện các mẫu hoạt động bất thường hoặc phân cụm các trạng thái hoạt động của hệ thống.

Ngoài ra, trong lĩnh vực tài chính, thuật toán có thể hỗ trợ phát hiện gian lận giao dịch theo thời gian thực bằng cách nhận diện các điểm dữ liệu bất thường so với hành vi thông thường.

Trong bài toán giao thông thông minh, Incremental DBSCAN có thể được áp dụng để phân tích luồng giao thông theo thời gian, phát hiện các điểm ùn tắc hoặc các mẫu di chuyển bất thường.

Tổng thể, với khả năng xử lý dữ liệu liên tục và hiệu quả, Incremental DBSCAN là một công cụ mạnh mẽ cho các hệ thống hiện đại, nơi dữ liệu không ngừng thay đổi và yêu cầu phản ứng nhanh.

TÀI LIỆU THAM KHẢO

1. Nguyễn, A. T. (2026). Học không giám sát [Slide bài giảng]. Trường Đại học Kinh tế Thành phố Hồ Chí Minh.
2. Ester, M., Kriegel, H. P., Sander, J., & Xu, X. (1996, August). A density-based algorithm for discovering clusters in large spatial databases with noise. In *kdd* (Vol. 96, No. 34, pp. 226-231).
3. Ester, M., Kriegel, H. P., Sander, J., Wimmer, M., & Xu, X. Incremental clustering for mining in a data ware housing. University of Munich Oettingenstr, 67.
4. Bentley, J. L. (1975). Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9), 509-517.
5. Rousseeuw, P. J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics*, 20, 53-65.
6. Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.
7. Tan, P. N., Steinbach, M., & Kumar, V. (2019). *Introduction to Data Mining* (2nd ed.). Pearson.
8. Gan, G., Ma, C., & Wu, J. (2020). *Data Clustering: Theory, Algorithms, and Applications*. SIAM.
9. Bentley, J. L. (1975). *Multidimensional binary search trees used for associative searching*. *Communications of the ACM*, 18(9), 509–517.

10. Schubert, E., Sander, J., Ester, M., Kriegel, H. P., & Xu, X. (2017). *DBSCAN revisited, revisited: Why and how you should (still) use DBSCAN*. ACM Transactions on Database Systems (TODS), 42(3), 1–21.
11. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
12. Davies, D. L., & Bouldin, D. W. (2009). A cluster separation measure. IEEE transactions on pattern analysis and machine intelligence, (2), 224-227

BẢNG PHÂN CÔNG NHIỆM VỤ

Thành viên	Nhiệm vụ	Mức độ hoàn thành
Nguyễn Ngọc Tú Trinh	I. Lời mở đầu II. Cơ sở lý thuyết Chọn dataset và tiền xử lý dữ liệu Code các thuật toán phân cụm khác trên dataset (Kmeans - HAC) Slide thuyết trình	100%
Hoàng Huy Tùng	III. Trình bày các bước giải bài toán đơn giản Code thuật toán DBSCAN Slide thuyết trình	100%
Trần Quốc Tuấn	IV. Triển khai thuật toán Incremental DBSCAN vào bộ dữ liệu thực tế Code thuật toán incremental DBSCAN Format hoàn chỉnh bài báo cáo	100%
Nguyễn Anh Tú	V. So sánh Incremental DBSCAN với các thuật toán VI. Kết luận và hướng phát triển Code đánh giá các thuật toán Format phần tiêu đề, hình ảnh, bảng biểu	100%